

LERNUNTERLAGE • INFORMATIKER EFZ • CLEAN CODE EDITION

Tutorial: Einfache WebApp mit Next.js, C# OData & PostgreSQL

Vollständig überarbeitete Ausgabe — Clean Code Edition

Next.js 14

C# OData

PostgreSQL 16

TypeScript

Docker

Version 2.0

September 2026

2. Lehrjahr

Inhaltsverzeichnis

1. Einführung & Überblick	3	5. Frontend: Next.js	
1.1 Was wird gebaut?	3	5.1 Projekt erstellen	
1.2 Tech-Stack-Überblick	4	5.2 API-Helper & Typen	
1.3 Voraussetzungen	5	5.3 Adressliste (GET)	
1.4 VS Code Extensions	6	5.4 ConfirmDialog-Komponente (F-03)	
2. Projektstruktur & Setup	7	5.5 AdresseForm: Controlled Inputs (F-02)	
2.1 Ordnerstruktur	7	5.6 NextAuth Session-Augmentation (F-04)	
2.2 Installation der Tools	9	6. Zusammenspiel testen	
2.3 VS Code Workspace	11	6.1 Starten & Testen	
3. Datenbank: PostgreSQL	12	6.2 Häufige Fehler	
3.1 PostgreSQL starten	13	7. Dockerisierung	
3.2 Datenbank erstellen	15	7.1 Was ist Docker?	
3.3 Tabelle Adressen	16	7.2 Dockerfiles	
3.4 Testdaten einfügen	17	7.3 docker-compose.yml	
4. Backend: ASP.NET Core mit OData	18	7.4 Ausführen & Testen	
4.1 Projekt erstellen	19	8. Stadt-Entität & PLZ-Wertehilfe	
4.2 NuGet-Pakete	19	8.1 City-Model & Migration	
4.3 Model & DbContext	20	8.2 CitiesController (OData CRUD)	
4.4 EF Core Migrations	23	8.3 Frontend: Typen & API-Funktionen	
4.5 OData Konfiguration	24	8.4 CityForm & Städte-Seiten	
4.6 Generische Basisklasse (DRY)	26	8.5 PLZ-Autocomplete im Adressformular	
4.7 DbSeeder: Seed-Logik (SRP)		9. Benutzerverwaltung & Login	
4.8 DTOs in eigenen Dateien (SRP)		9.1 Backend: User-Model & AuthController	
4.9 CORS konfigurieren		9.2 Passwort-Hashing (ASP.NET Identity)	
4.10 Backend testen		9.3 Frontend: NextAuth.js einrichten	

9.4 Google OAuth integrieren	12.2 Backend: AuditableEntity Basisklasse
9.5 Login-Seite & Route-Schutz	12.3 DbContext: automatisches Befüllen
10. E-Mail-Benachrichtigung & Einstellungen	12.4 Migration: Spalten hinzufügen
10.1 Backend: EmailService (SMTP)	12.5 Frontend: TypeScript-Typen
10.2 Backend: Settings-Entität & Controller	13. Häufige Fehler & Debugging
10.3 Frontend: Einstellungen-Seite	13.1 Seed-Benutzer schlägt fehl (NOT NULL)
11. Sicherheitsüberlegungen	13.2 NextAuth: OAuthSignin-Fehler & 401
11.1 Timing-Angriffe abwehren	13.3 Google-Button bedingt anzeigen
11.2 User-Enumeration verhindern	13.4 Docker: Volume & Browser-Cache
11.3 Secrets & Umgebungsvariablen	14. Clean Code & Refactoring
12. Audit-Felder & Gültigkeitszeitraum	14.1 Was ist Clean Code?
12.1 Warum Audit-Trails?	14.2 Übersicht: 14 Befunde
	14.3 Refactoring-Workflow

KAPITEL 1

1. Einführung & Überblick

Lernziele dieses Kapitels

- Du weißt, was wir in diesem Tutorial bauen und welchen Nutzen die Applikation hat.
- Du kennst die verwendeten Technologien und deren Zusammenspiel.
- Du hast alle notwendigen Werkzeuge und Extensions auf deinem Rechner installiert.
- Du verstehst, was OData ist und warum es für Web-APIs sinnvoll ist.

1.1 Was wird gebaut?

In diesem Tutorial erstellen wir gemeinsam eine vollständige **Adressverwaltungs-Webapplikation** mit CRUD-Funktionalität (Create, Read, Update, Delete). Die Applikation ermöglicht es, Personenadressen zu erfassen, anzuzeigen, zu bearbeiten und zu löschen.

Eine Adresse besteht aus folgenden Feldern:

Feld	Datentyp	Beschreibung	Beispiel
------	----------	--------------	----------

Id	integer	Primärschlüssel, automatisch	1, 2, 3 ...
Vorname	string	Vorname der Person	Anna
Name	string	Nachname der Person	Meier
Strasse	string	Strassenname	Bahnhofstrasse
Strassennummer	string	Hausnummer (inkl. Zusatz)	12a
Plz	string	Postleitzahl	8001
Ort	string	Ortsname	Zürich

Das Ergebnis ist eine moderne, dreischichtige Webapplikation (3-Tier-Architektur): Das **Frontend** läuft im Browser des Benutzers, das **Backend** stellt eine REST-API bereit, und die **Datenbank** speichert alle Daten dauerhaft. Alle drei Schichten kommunizieren über klar definierte Schnittstellen miteinander.

Tipp: CRUD steht für Create, Read, Update, Delete

Diese vier Grundoperationen bilden das Rückgrat jeder datenverwaltenden Applikation. Du wirst sie in diesem Tutorial vollständig implementieren: Adressen anlegen (Create), auflisten und anzeigen (Read), bearbeiten (Update) und löschen (Delete).

1.2 Tech-Stack-Überblick

Wir setzen auf einen modernen, in der Industrie weit verbreiteten Tech-Stack. Jede Technologie hat eine klar definierte Aufgabe:

Schicht	Technologie	Version	Aufgabe
Frontend	Next.js (React)	14+	Benutzerinterface im Browser, Formulare, Listen
Styling	Tailwind CSS	3	Utility-basiertes CSS-Framework für modernes Design
Backend	ASP.NET Core Web API	8 (LTS)	REST-API-Server in C#, verarbeitet HTTP-Anfragen
API-Standard	OData v4	8.x	Standardisiertes Abfrageprotokoll für REST-APIs
ORM	Entity Framework Core	8	Datenbankzugriff in C#, Code-First-Migrationen
Datenbank	PostgreSQL	16	Relationale Open-Source-Datenbank
DB-Treiber	Npgsql	8	PostgreSQL-Provider für Entity Framework Core

Was ist OData?

OData (Open Data Protocol) ist ein offener Standard des *OASIS*-Konsortiums, der auf REST und HTTP aufbaut. OData definiert, wie eine API über URLs strukturiert werden soll, und ermöglicht standardisierte Abfragen direkt in der URL:

OData-Abfrage	Bedeutung
<code>GET /odata/Adressen</code>	Alle Adressen abrufen
<code>GET /odata/Adressen(5)</code>	Adresse mit Id=5 abrufen
<code>GET /odata/Adressen?\$filter=Ort eq 'Zürich'</code>	Nur Zürcher Adressen
<code>GET /odata/Adressen?\$orderby=Name</code>	Alphabetisch sortieren
<code>GET /odata/Adressen?\$top=10&\$skip=20</code>	Seite 3 (je 10 Einträge)
<code>GET /odata/Adressen?\$select=Vorname, Name</code>	Nur bestimmte Felder

Der Vorteil: Das Frontend muss keine eigenen Filtermechanismen implementieren. Es nutzt einfach die OData-URL-Syntax, und das Backend erledigt den Rest automatisch – inklusive Datenbankoptimierung.

Architektur-Übersicht

Browser (Next.js) → HTTP/OData → ASP.NET Core Web API → Entity Framework Core → PostgreSQL

Jede Schicht kommuniziert nur mit der direkt benachbarten Schicht. Das Backend kennt das Frontend nicht, und das Frontend kennt die Datenbank nicht.

1.3 Voraussetzungen

Vor Beginn müssen folgende Tools auf deinem System installiert sein. In Kapitel 2 wird die Installation Schritt für Schritt erklärt.

Tool	Mindestversion	Zweck	Prüfbefehl
Node.js	18 LTS	Laufzeitumgebung für Next.js	<code>node --version</code>
npm	9	Paketmanager für JavaScript	<code>npm --version</code>
.NET SDK	8.0	Compiler und Laufzeit für C#	<code>dotnet --version</code>

PostgreSQL	15 oder 16	Datenbankserver	<code>psql --version</code>
Git	2.40	Versionsverwaltung	<code>git --version</code>
Docker Desktop	4.25	Containerisierung (Kapitel 7)	<code>docker --version</code>

Achtung: .NET SDK vs. Runtime

Für die Entwicklung benötigst du das **.NET SDK 8** (nicht nur die Runtime). Das SDK enthält Compiler, Paketmanager und die `dotnet`-CLI. Prüfe mit `dotnet --list-sdks`, ob Version 8.x vorhanden ist.

1.4 Empfohlene VS Code Extensions

Visual Studio Code ist der empfohlene Editor für dieses Tutorial. Folgende Extensions erleichtern die Entwicklung erheblich:

Extension	ID	Nutzen
C# Dev Kit	<code>ms-dotnettools.csdevkit</code>	IntelliSense, Debugging, Refactoring für C#
C# (OmniSharp)	<code>ms-dotnettools.csharp</code>	Wird von C# Dev Kit automatisch installiert
ESLint	<code>dbaeumer.vscode-eslint</code>	JavaScript/TypeScript Linting
Prettier	<code>esbenp.prettier-vscode</code>	Automatische Code-Formatierung
PostgreSQL	<code>cweijan.vscode-postgresql-client2</code>	Datenbankverbindung und SQL-Ausführung in VS Code
REST Client	<code>humao.rest-client</code>	HTTP-Anfragen direkt in VS Code testen
Tailwind IntelliSense	<code>bradlc.vscode-tailwindcss</code>	Autovervollständigung für Tailwind-Klassen
Docker	<code>ms-azuretools.vscode-docker</code>	Dockerfile-Support und Container-Verwaltung

Du kannst alle Extensions auf einmal installieren, indem du die Extension-IDs in der VS Code Extensions-Suche eingibst. In Kapitel 2 richten wir ausserdem eine `extensions.json`-Datei ein, die VS Code beim Öffnen des Projekts automatisch zur Installation der empfohlenen Extensions auffordert.

Tipp: Extensions via Kommandozeile installieren

Mit dem Befehl `code --install-extension <ID>` lässt sich jede Extension direkt über das Terminal installieren, ohne die VS Code GUI zu öffnen. Besonders praktisch bei der Ersteinrichtung eines neuen Arbeitsplatzes.

Zusammenfassung Kapitel 1

- Wir bauen eine **Adressverwaltungs-WebApp** mit den Feldern Vorname, Name, Strasse, Strassennummer, PLZ und Ort.
- Der Tech-Stack umfasst **Next.js** (Frontend), **ASP.NET Core 8 mit OData** (Backend) und **PostgreSQL** (Datenbank).
- **OData** ist ein standardisiertes Protokoll für REST-APIs mit mächtigen URL-basierten Abfragemöglichkeiten.
- Die empfohlenen VS Code Extensions erleichtern die Entwicklung in allen drei Technologiebereichen.

KAPITEL 2

2. Projektstruktur & Setup

Lernziele dieses Kapitels

- Du kennst die Ordnerstruktur des Gesamtprojekts und weisst, wo welche Dateien hingehören.
- Du hast Node.js, das .NET SDK 8 und PostgreSQL lokal installiert und geprüft.
- Du hast einen VS Code Workspace eingerichtet mit allen empfohlenen Extensions.
- Du verstehst, warum eine klare Projektstruktur wichtig ist.

2.1 Ordnerstruktur des Gesamtprojekts

Das gesamte Projekt liegt in einem einzigen Wurzelordner `adressverwaltung/`. Darin befinden sich drei Unterordner für die drei Schichten sowie die Docker Compose-Konfiguration im Wurzelverzeichnis.

```
adressverwaltung/  
├─ backend/# ASP.NET Core Web API (C#)  
│ └─ Controllers/  
│   └─ AdressenController.cs  
│ └─ Models/
```

```

| | └─ AdressenController.cs
| └─ Models/
| | └─ Adresse.cs
| └─ Data/
| | └─ AdresseDbContext.cs
| └─ Migrations/# Autogenerated von EF Core
| └─ AdressverwaltungApi.csproj
| └─ Program.cs
| └─ appsettings.json
| └─ appsettings.Development.json
| └─ Dockerfile
└─ frontend/# Next.js (TypeScript)
  └─ app/# Next.js App Router
    └─ layout.tsx
      └─ page.tsx# Adressliste (Startseite)
        └─ adressen/
          └─ neu/
            └─ page.tsx# Neue Adresse erstellen
              └─ [id]/bearbeiten/
                └─ page.tsx# Adresse bearbeiten
          └─ components/
            └─ AdresseForm.tsx# Wiederverwendbares Formular
          └─ lib/
            └─ api.ts# OData API-Helper-Funktionen
          └─ types/
            └─ adresse.ts# TypeScript-Typdefinition
        └─ next.config.ts
      └─ tailwind.config.ts
    └─ package.json
    └─ Dockerfile
  └─ .vscode/# VS Code Workspace-Einstellungen
    └─ extensions.json
    └─ settings.json
  └─ docker-compose.yml# Alle drei Services

```

Tipp: Ordner mit dem Terminal anlegen

Erstelle die Grundstruktur mit einem einzigen Befehl im Terminal:

```
bash
```

```

mkdir -p adressverwaltung/{backend,frontend,.vscode}
cd adressverwaltung

```

2.2 Installation der Tools

Node.js installieren

Node.js ist die JavaScript-Laufzeitumgebung, auf der Next.js basiert. Empfohlen wird die LTS-Version (Long Term Support).

Windows & macOS: Lade den Installer von nodejs.org herunter und führe ihn aus. npm wird automatisch mitinstalliert.

Linux (Ubuntu/Debian):

```
bash
```

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -  
sudo apt-get install -y nodejs
```

Überprüfe die Installation:

```
bash
```

```
node --version    # Erwartet: v20.x.x oder höher  
npm --version     # Erwartet: 10.x.x oder höher
```

.NET SDK 8 installieren

Das .NET SDK 8 enthält alles, was du für die C#-Entwicklung brauchst: Compiler, Laufzeit, NuGet-Paketmanager und die [dotnet](https://dotnet.microsoft.com/cli) -CLI.

Windows & macOS: Lade den SDK-Installer von dotnet.microsoft.com/download herunter. Wähle explizit das SDK (nicht nur die Runtime).

Linux (Ubuntu):

```
bash
```

```
wget https://packages.microsoft.com/config/ubuntu/22.04/packages-microsoft-prod.deb  
sudo dpkg -i packages-microsoft-prod.deb  
sudo apt-get update  
sudo apt-get install -y dotnet-sdk-8.0
```

Überprüfe die Installation und installiere das EF Core-Tool:

```
bash
```

```
dotnet --version      # Erwartet: 8.0.x
dotnet --list-sdks     # Zeigt alle installierten SDKs

# EF Core CLI-Tool global installieren (wird für Migrationen gebraucht)
dotnet tool install --global dotnet-ef
dotnet ef --version    # Erwartet: 8.x.x
```

Achtung: dotnet-ef muss zum PATH hinzugefügt werden

Nach der Installation von `dotnet-ef` füge das globale Tool-Verzeichnis zu deinem PATH hinzu. Auf Linux/macOS: `export PATH="$PATH:$HOME/.dotnet/tools"` in deine `~/.bashrc` oder `~/.zshrc` eintragen. Auf Windows geschieht dies automatisch.

PostgreSQL installieren

PostgreSQL ist der Datenbankserver, der unsere Adressdaten speichert. Du benötigst den Server selbst sowie das Client-Tool `psql`.

Windows: Lade den Installer von postgresql.org/download/windows herunter. Der Installer enthält Server, psql und pgAdmin 4.

macOS (mit Homebrew):

```
bash
```

```
brew install postgresql@16
brew services start postgresql@16
```

Linux (Ubuntu):

```
bash
```

```
sudo apt-get install -y postgresql postgresql-client
sudo systemctl start postgresql
sudo systemctl enable postgresql # Autostart beim Systemstart
```

Überprüfe die Installation:

```
bash
```

```
psql --version # Erwartet: psql (PostgreSQL) 15.x oder 16.x
```

2.3 VS Code Workspace einrichten

Wir richten einen VS Code Workspace ein, der automatisch die empfohlenen Extensions vorschlägt und konsistente Code-Formatierung sicherstellt. Erstelle die folgenden zwei Dateien im Ordner `adressverwaltung/.vscode/`:

Datei: `.vscode/extensions.json`

Diese Datei teilt VS Code mit, welche Extensions für dieses Projekt empfohlen werden. Beim ersten Öffnen des Ordners in VS Code erscheint eine Aufforderung zur Installation.

```
.vscode/extensions.json
{
  "recommendations": [
    "ms-dotnettools.csdevkit",
    "ms-dotnettools.csharp",
    "dbaeumer.vscode-eslint",
    "esbenp.prettier-vscode",
    "cweijan.vscode-postgresql-client2",
    "humao.rest-client",
    "bradlc.vscode-tailwindcss",
    "ms-azuretools.vscode-docker"
  ]
}
```

Datei: `.vscode/settings.json`

Diese Datei definiert projektweite Editor-Einstellungen, die für alle Teammitglieder gelten. Damit wird sichergestellt, dass Code immer gleich formatiert wird – unabhängig davon, welche persönlichen VS Code-Einstellungen jemand hat.

```
.vscode/settings.json
```

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "[csharp]": {
    "editor.defaultFormatter": "ms-dotnettools.csdevkit"
  },
  "editor.tabSize": 2,
  "editor.insertSpaces": true,
  "files.trimTrailingWhitespace": true,
  "files.insertFinalNewline": true,
  "prettier.singleQuote": true,
  "prettier.semi": true,
  "typescript.preferences.importModuleSpecifier": "relative"
}
```

Öffne nun den gesamten Projektordner in VS Code:

```
bash
```

```
code adressverwaltung/
```

VS Code öffnet sich und fragt, ob die empfohlenen Extensions installiert werden sollen. Bestätige mit **Alle installieren**. Ab jetzt hast du IntelliSense, automatische Formatierung und alle weiteren Hilfsmittel direkt im Editor verfügbar.

Tipp: Integriertes Terminal in VS Code

Öffne das integrierte Terminal mit **Ctrl+`** (Windows/Linux) oder **Cmd+`** (macOS). Du kannst mehrere Terminals gleichzeitig öffnen – sehr praktisch, wenn Backend und Frontend parallel laufen sollen.

Zusammenfassung Kapitel 2

- Das Projekt ist klar in drei Ordner aufgeteilt: `backend/`, `frontend/` und `.vscode/`.
- Node.js 20 LTS, .NET SDK 8, dotnet-ef (global) und PostgreSQL 16 sind installiert und überprüft.
- Der VS Code Workspace ist mit `extensions.json` und `settings.json` konfiguriert.
- Mit **Format on Save** wird Code automatisch beim Speichern formatiert.

KAPITEL 3

3. Datenbank: PostgreSQL

Lernziele dieses Kapitels

- Du kannst PostgreSQL lokal starten und dich mit dem Client-Tool `psql` verbinden.
- Du kannst eine Datenbank und eine Tabelle mit SQL anlegen.
- Du verstehst den Unterschied zwischen dem Anlegen der Datenbank per SQL und per EF Core Migration.
- Du kannst Testdaten in die Tabelle einfügen und abfragen.

In diesem Kapitel richten wir die Datenbankschicht ein. PostgreSQL ist ein leistungsstarkes, quelloffenes, relationales Datenbanksystem. Es wird weltweit in produktiven Umgebungen eingesetzt und unterstützt alle SQL-Standards vollständig.

Hinweis: Zwei Wege zur Datenbankvorbereitung

Es gibt zwei Vorgehensweisen: (1) Datenbank und Tabellen per Hand mit SQL anlegen (wie in diesem Kapitel gezeigt), oder (2) nur die Datenbank anlegen und dann EF Core Migrations die Tabellen erstellen lassen (Kapitel 4). Für das Verständnis zeigen wir beide Wege. Im produktiven Einsatz ist der Migrations-Ansatz vorzuziehen.

3.1 PostgreSQL starten und verbinden

Nach der Installation aus Kapitel 2 muss der PostgreSQL-Dienst laufen. Überprüfe den Status und starte ihn bei Bedarf:

```
bash - Linux/macOS
```

```
# Status prüfen
sudo systemctl status postgresql # Linux
brew services list                # macOS

# Starten (falls nicht aktiv)
sudo systemctl start postgresql  # Linux
brew services start postgresql@16 # macOS
```

```
bash - Windows (PowerShell als Administrator)
```

```
# PostgreSQL-Dienst starten
Start-Service postgresql-x64-16

# Oder: Dienste-Manager öffnen
services.msc
```

Mit psql verbinden

`psql` ist das offizielle Kommandozeilen-Client-Tool für PostgreSQL. Verbinde dich mit dem Standardbenutzer `postgres`:

```
bash

# Linux: Zuerst zum postgres-Systembenutzer wechseln
sudo -u postgres psql

# macOS / Windows: Direkt verbinden
psql -U postgres -h localhost
```

Du siehst jetzt den psql-Prompt: `postgres=#`. Hier kannst du SQL-Befehle eingeben. Nützliche psql-Meta-Befehle:

Befehl	Bedeutung
<code>\l</code>	Alle Datenbanken auflisten
<code>\c dbname</code>	Mit Datenbank verbinden
<code>\dt</code>	Alle Tabellen in aktueller Datenbank
<code>\d tablename</code>	Struktur einer Tabelle anzeigen
<code>\q</code>	psql beenden
<code>\?</code>	Alle Meta-Befehle anzeigen

Tipp: pgAdmin 4 als grafische Alternative

pgAdmin 4 ist eine grafische Benutzeroberfläche für PostgreSQL und wird beim Windows-Installer automatisch mitinstalliert. Sie ermöglicht das Ausführen von SQL-Abfragen, das Durchsuchen von Tabellen und die Verwaltung von Datenbankbenutzern – alles ohne Kommandozeile. Auf macOS und Linux kann pgAdmin separat installiert werden.

Passwort für den postgres-Benutzer setzen

Im psql-Prompt setzt du ein Passwort für den `postgres`-Benutzer. Dieses Passwort wird später im Connection String des Backends benötigt:

```
sql - psql-Prompt
```

```
ALTER USER postgres WITH PASSWORD 'deinPasswort123';  
-- Erwartet: ALTER ROLE
```

Achtung: Sicherheit bei Produktionssystemen

Für dieses Tutorial verwenden wir den `postgres`-Superuser zur Vereinfachung. In einem produktiven System solltest du einen dedizierten Datenbankbenutzer mit eingeschränkten Rechten erstellen und niemals das Superuser-Passwort in einer Konfigurationsdatei speichern, die in einem Repository eingecheckt wird.

3.2 Datenbank erstellen

Erstelle eine neue Datenbank mit dem Namen `adressverwaltung`. Diese Datenbank wird ausschliesslich für unser Projekt verwendet.

```
sql - psql-Prompt (als postgres-Benutzer)
```

```
-- Datenbank erstellen  
CREATE DATABASE adressverwaltung  
    WITH ENCODING 'UTF8'  
    LC_COLLATE = 'de_CH.UTF-8'  
    LC_CTYPE   = 'de_CH.UTF-8'  
    TEMPLATE   = template0;  
  
-- Überprüfung  
\l adressverwaltung
```

Achtung: Locale auf Windows und macOS

Wenn `de_CH.UTF-8` nicht verfügbar ist, verwende stattdessen `TEMPLATE = template1` und lasse `LC_COLLATE` und `LC_CTYPE` weg. Die Datenbank wird dann mit den Systemstandards erstellt.

Verbinde dich mit der neuen Datenbank:

```
sql - psql-Prompt
```

```
\c adressverwaltung
-- Erwartet: You are now connected to database "adressverwaltung" as user "postgres".
```

3.3 Tabelle Adressen erstellen

Erstelle die Tabelle `"Adressen"` (mit Anführungszeichen, damit der Name gross geschrieben bleibt). Die Tabelle enthält alle sieben Felder unserer Adresse:

```
sql - adressverwaltung-Datenbank

-- Tabelle erstellen (mit "serial" für automatischen Primärschlüssel)
CREATE TABLE IF NOT EXISTS "Adressen" (
    "Id"          SERIAL          PRIMARY KEY,
    "Vorname"     VARCHAR(100)    NOT NULL,
    "Name"        VARCHAR(100)    NOT NULL,
    "Strasse"     VARCHAR(200)    NOT NULL,
    "Strassennummer" VARCHAR(20)  NOT NULL,
    "Plz"         VARCHAR(10)     NOT NULL,
    "Ort"         VARCHAR(100)    NOT NULL
);

-- Überprüfung
\d "Adressen"
```

Erklärung der SQL-Schlüsselwörter:

- `SERIAL` — Ganzzahl, die automatisch um 1 erhöht wird (entspricht `INTEGER GENERATED ALWAYS AS IDENTITY`).
- `PRIMARY KEY` — Jeder Wert muss eindeutig und nicht NULL sein. Automatisch wird ein Index erstellt.
- `VARCHAR(n)` — Zeichenkette mit maximaler Länge `n`.
- `NOT NULL` — Das Feld muss einen Wert enthalten; NULL ist nicht erlaubt.
- `IF NOT EXISTS` — Kein Fehler, wenn die Tabelle bereits existiert.

Tipp: Anführungszeichen in PostgreSQL

PostgreSQL konvertiert Bezeichner (Tabellen-, Spaltennamen) standardmässig in Kleinbuchstaben. Um die Grossschreibung zu erhalten – wie EF Core sie erwartet – schreiben wir die Namen in doppelte Anführungszeichen: `"Adressen"`, `"Id"`, etc. Das ist wichtig für die Kompatibilität mit dem C#-Code.

3.4 Testdaten einfügen und abfragen

Füge einige Testdatensätze ein, damit du das Backend direkt testen kannst, ohne zuerst das Frontend fertigzustellen:

sql - INSERT-Befehle

```
INSERT INTO "Adressen" ("Vorname", "Name", "Strasse", "Strassennummer", "Plz", "Ort")
VALUES
  ('Anna', 'Meier', 'Bahnhofstrasse', '12', '8001', 'Zürich'),
  ('Hans', 'Müller', 'Hauptgasse', '5a', '3011', 'Bern'),
  ('Sophie', 'Schmid', 'Seestrasse', '88', '6004', 'Luzern'),
  ('Tobias', 'Koch', 'Marktgasse', '3', '9000', 'St. Gallen'),
  ('Laura', 'Brunner', 'Rosenweg', '17b', '4051', 'Basel');

-- Überprüfung
SELECT * FROM "Adressen";
```

Die Abfrage gibt alle eingefügten Datensätze zurück. Die `Id`-Spalte wurde automatisch von der SERIAL-Sequenz befüllt (1 bis 5).

Weitere nützliche Abfragen zum Testen:

sql - Weitere Abfragen

```
-- Adressen aus Zürich
SELECT "Vorname", "Name", "Ort"
FROM "Adressen"
WHERE "Ort" = 'Zürich';

-- Nach Nachnamen sortiert
SELECT * FROM "Adressen"
ORDER BY "Name" ASC;

-- Anzahl Einträge
SELECT COUNT(*) FROM "Adressen";

-- Einzelnen Datensatz per ID abrufen
SELECT * FROM "Adressen" WHERE "Id" = 1;
```

Hinweis: EF Core erstellt die Tabelle nochmals

In Kapitel 4 werden wir EF Core Migrations verwenden. Diese erstellen die Tabelle programmatisch auf Basis des C#-Modells. Wenn du die Tabelle bereits per SQL erstellt hast, musst du sie entweder vorher löschen (`DROP TABLE "Adressen"`), oder die Migration so anpassen, dass sie eine bestehende Tabelle

nicht nochmals erstellt. Für das Tutorial empfehlen wir: Tabelle per SQL erstellen (wie oben), die EF Core Migration aber trotzdem durchführen – EF Core prüft, ob die Tabelle bereits existiert.

Verbindung über die VS Code Extension testen

Mit der installierten PostgreSQL-Extension (cweijan) kannst du dich direkt aus VS Code mit der Datenbank verbinden:

1. Öffne die Datenbankansicht in VS Code (Elefant-Icon in der Seitenleiste).
2. Klicke auf + um eine neue Verbindung hinzuzufügen.
3. Fülle die Felder aus: Host = `localhost` , Port = `5432` , Benutzer = `postgres` , Passwort = dein Passwort, Datenbank = `adressverwaltung` .
4. Klicke auf **Connect**. Du siehst jetzt die Tabellenstruktur und kannst SQL-Abfragen direkt in VS Code ausführen.

Zusammenfassung Kapitel 3

- PostgreSQL läuft lokal und ist über `psql` und die VS Code Extension erreichbar.
- Die Datenbank `adressverwaltung` und die Tabelle `"Adressen"` mit 7 Feldern sind erstellt.
- Fünf Testdatensätze sind vorhanden und können abgefragt werden.
- Anführungszeichen in SQL sichern die Grossschreibung von Tabellen- und Spaltennamen.

KAPITEL 4

4. Backend: ASP.NET Core mit OData

Lernziele dieses Kapitels

- Du kannst ein ASP.NET Core Web API-Projekt erstellen und konfigurieren.
- Du verstehst das Konzept von Entity Framework Core und kannst ein Model und einen DbContext erstellen.
- Du kannst EF Core Migrations erstellen und auf die Datenbank anwenden.
- Du kannst OData in ASP.NET Core konfigurieren und einen vollständigen CRUD-Controller implementieren.
- Du kannst CORS konfigurieren und das Backend mit dem REST Client testen.

4.1 Neues Projekt erstellen

Wechsle in das Backend-Verzeichnis und erstelle ein neues ASP.NET Core Web API-Projekt mit dem `dotnet`-CLI:

```
bash
```

```
cd adressverwaltung/backend

dotnet new webapi -n AdressverwaltungApi --no-https

# Ins Projektverzeichnis wechseln
cd AdressverwaltungApi
```

Tipp: --no-https für die lokale Entwicklung

Das Flag `--no-https` deaktiviert die HTTPS-Umleitung für die lokale Entwicklung. Das vereinfacht das Setup erheblich, da kein lokales SSL-Zertifikat benötigt wird. In einer Produktionsumgebung würde man HTTPS aktivieren und ein gültiges Zertifikat verwenden.

Starte das Projekt zum ersten Mal, um sicherzustellen, dass alles funktioniert:

```
bash
```

```
dotnet run

# Erwartet: Now listening on: http://localhost:5000 (oder ähnliche URL)
```

Öffne `http://localhost:5000/weatherforecast` im Browser. Du siehst eine JSON-Antwort mit Beispieldaten – das Standardprojekt funktioniert. Stoppe den Server wieder mit **Ctrl+C**.

4.2 NuGet-Pakete installieren

Wir benötigen drei NuGet-Pakete: OData-Integration, Entity Framework Core und den PostgreSQL-Treiber:

```
bash
```

```
# OData-Support für ASP.NET Core
dotnet add package Microsoft.AspNetCore.OData --version 8.2.5

# EF Core mit PostgreSQL-Provider (Npgsql)
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL --version 8.0.4

# EF Core Design-Tools (für Migrationen)
dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.7
```

Überprüfe, ob alle Pakete korrekt in der Projektdatei eingetragen wurden:

```
bash
```

```
dotnet restore
# Erwartet: Restore completed
```

Was macht jedes Paket?

- **Microsoft.AspNetCore.OData** — Fügt OData-Routing, -Abfragen und -Serialisierung zum Web API hinzu.
- **Npgsql.EntityFrameworkCore.PostgreSQL** — EF Core Treiber für PostgreSQL. Übersetzt LINQ-Abfragen in PostgreSQL-SQL.
- **Microsoft.EntityFrameworkCore.Design** — Wird zur Entwicklungszeit für Migration-Generierung benötigt. Nicht im Production-Build enthalten.

4.3 Model und DbContext erstellen

Model: Adresse.cs

Das Model ist eine einfache C#-Klasse, die eine Zeile in der Datenbanktabelle repräsentiert. Erstelle den Ordner `Models/` und die Datei `Adresse.cs` :

```
Models/Adresse.cs
```

```

using System.ComponentModel.DataAnnotations;

namespace AdressverwaltungApi.Models;

public class Adresse : AuditableEntity
{
    public int Id { get; set; }

    [Required]
    [MaxLength(100)]
    public string Vorname { get; set; } = string.Empty;

    [Required]
    [MaxLength(100)]
    public string Name { get; set; } = string.Empty;

    [Required]
    [MaxLength(200)]
    public string Strasse { get; set; } = string.Empty;

    [Required]
    [MaxLength(20)]
    public string Strassennummer { get; set; } = string.Empty;

    [Required]
    [MaxLength(10)]
    public string Plz { get; set; } = string.Empty;

    [Required]
    [MaxLength(100)]
    public string Ort { get; set; } = string.Empty;
}

```

Doppelte Deklaration — warum?

In `OnModelCreating` werden die Constraints für EF Core (SQL-Ebene) deklariert. Die `[Required]` - und `[MaxLength]` -Attribute auf dem Modell aktivieren zusätzlich die serverseitige Modell-Validierung via `ModelState` (HTTP-Ebene). Beide Layer schützen unterschiedliche Pfade: `DataAnnotations` fangen ungültige API-Anfragen ab, `OnModelCreating` legt die Datenbankstruktur fest.

Wichtige Konzepte:

- `{ get; set; }` — C#-Property mit automatisch generiertem Getter und Setter.
- `= string.Empty` — Initialisiert die Property mit einem leeren String, damit der Null-Safety-Compiler zufrieden ist.
- EF Core erkennt `Id` automatisch als Primärschlüssel (Konvention).

- `[Required]` und `[MaxLength]` aktivieren die HTTP-Ebene-Validierung via `ModelState`.

DbContext: AdresseDbContext.cs

Der `DbContext` ist die Brücke zwischen dem C#-Code und der Datenbank. Er definiert, welche Tabellen es gibt und wie sie konfiguriert sind. Erstelle den Ordner `Data/` und die Datei `AdresseDbContext.cs`:

Data/AdresseDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using AdressverwaltungApi.Models;

namespace AdressverwaltungApi.Data;

public class AdresseDbContext : DbContext
{
    public AdresseDbContext(DbContextOptions<AdresseDbContext> options)
        : base(options)
    {
    }

    // Repräsentiert die Tabelle "Adressen" in der Datenbank
    public DbSet<Adresse> Adressen { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Adresse>(entity =>
        {
            // Tabellenname in PostgreSQL (gross geschrieben, mit Anführungszeichen)
            entity.ToTable("Adressen");

            // Primärschlüssel
            entity.HasKey(a => a.Id);

            // Spalten-Konfiguration
            entity.Property(a => a.Vorname).IsRequired().HasMaxLength(100);
            entity.Property(a => a.Name).IsRequired().HasMaxLength(100);
            entity.Property(a => a.Strasse).IsRequired().HasMaxLength(200);
            entity.Property(a => a.Strassennummer).IsRequired().HasMaxLength(20);
            entity.Property(a => a.Plz).IsRequired().HasMaxLength(10);
            entity.Property(a => a.Ort).IsRequired().HasMaxLength(100);
        });
    }
}
```

Tipp: `DbSet<T>` = Tabelle

Jede `DbSet<T>` -Property im `DbContext` repräsentiert eine Datenbanktabelle. Der Name der Property (`Adressen`) entspricht standardmässig dem Tabellennamen. Mit `entity.ToTable("Adressen")` in `OnModelCreating` stellen wir sicher, dass der exakte Name (gross/klein) korrekt ist.

4.4 Connection String und EF Core Migrations

Connection String in `appsettings.json`

Öffne die Datei `appsettings.json` und füge den Connection String für PostgreSQL hinzu:

`appsettings.json`

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Host=localhost;Port=5432;Database=adressverwaltung;Username=postgres;Password=deinPasswort123"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Achtung: Passwort nicht ins Repository einchecken!

Füge `appsettings.json` (mit dem echten Passwort) zur `.gitignore` -Datei hinzu, oder verwende stattdessen `appsettings.Development.json` für lokale Credentials. In Produktionsumgebungen werden Credentials als Umgebungsvariablen oder über einen Secret Manager verwaltet.

EF Core Migrations erstellen und anwenden

EF Core Migrations generieren C#-Code, der die Datenbankstruktur anlegt oder ändert. Der Workflow ist:

1. Du änderst das C#-Model.
2. Du erstellst eine neue Migration (`dotnet ef migrations add`).
3. Du wendest die Migration auf die Datenbank an (`dotnet ef database update`).

```
bash - im Ordner backend/AdressverwaltungApi/
```

```
# Erste Migration erstellen (legt die Tabellenstruktur fest)
dotnet ef migrations add InitialCreate

# Migration auf die Datenbank anwenden
dotnet ef database update

# Überprüfung: Welche Migrationen wurden angewendet?
dotnet ef migrations list
```

Nach dem Ausführen findest du einen neuen Ordner `Migrations/` mit drei automatisch generierten Dateien. Die Migration erstellt die Tabelle `"Adressen"` in der Datenbank (wenn sie noch nicht existiert).

4.5 OData in Program.cs konfigurieren

Ersetze den Inhalt von `Program.cs` vollständig mit der folgenden Konfiguration. In Version 2.0 sind zusätzlich `DbSeeder` (SRP, Kapitel 4.7) und `INotificationService` (SRP, Kapitel 10) registriert:

```
Program.cs
```

```

using Microsoft.AspNetCore.OData;
using Microsoft.AspNetCore.Identity;
using Microsoft.OData.ModelBuilder;
using Microsoft.EntityFrameworkCore;
using AdressverwaltungApi;
using AdressverwaltungApi.Data;
using AdressverwaltungApi.Models;
using AdressverwaltungApi.Services;

var builder = WebApplication.CreateBuilder(args);

// === DATENBANK ===
builder.Services.AddDbContext<AdresseDbContext>(options =>
    options.UseNpgsql(
        builder.Configuration.GetConnectionString("DefaultConnection")
    )
);

// HTTP-Kontext (für Audit-Felder CreatedBy/ChangedBy)
builder.Services.AddHttpContextAccessor();

// Passwort-Hashing (PBKDF2 + Salt)
builder.Services.AddScoped<IPasswordHasher<User>, PasswordHasher<User>>();

// === SERVICES ===
builder.Services.AddScoped<IEmailService, EmailService>();
builder.Services.AddScoped<INotificationService, EmailNotificationService>();

// === ODATA: EDM-Modell ===
var modelBuilder = new ODataConventionModelBuilder();
modelBuilder.EnableLowerCamelCase();
modelBuilder.EntitySet<Adresse>("Adressen");
modelBuilder.EntitySet<City>("Cities");

// === CONTROLLERS + ODATA ===
builder.Services.AddControllers()
    .AddJsonOptions(opts =>
        opts.JsonSerializerOptions.PropertyNamingPolicy =
            System.Text.Json.JsonNamingPolicy.CamelCase)
    .AddOData(options => options
        .Select().Filter().OrderBy()
        .SetMaxTop(100).Count().Expand()
        .AddRouteComponents("odata", modelBuilder.GetEdmModel())
    );

// === CORS ===
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
        policy.WithOrigins("http://localhost:3000", "http://frontend:3000")
        .AllowAnyHeader().AllowAnyMethod());
});

```

```

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// === DATENBANK INITIALISIEREN ===
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AdresseDbContext>();
    var hasher = scope.ServiceProvider.GetRequiredService<IPasswordHasher<User>>();
    var config = scope.ServiceProvider.GetRequiredService<IConfiguration>();

    db.Database.EnsureCreated();
    DbSeeder.Seed(db, hasher, config); // ← ausgelagert (SRP, Kap. 4.7)
}

// === MIDDLEWARE ===
if (app.Environment.IsDevelopment()) { app.UseSwagger(); app.UseSwaggerUI(); }
app.UseCors("AllowFrontend");
app.UseRouting();
app.UseAuthorization();
app.MapControllers();
app.Run();

```

Was bedeuten die OData-Optionen?

- **Select()** — Erlaubt `?$select` : Nur bestimmte Felder zurückgeben.
- **Filter()** — Erlaubt `?$filter` : Serverseitige Filterung.
- **OrderBy()** — Erlaubt `?$orderby` : Serverseitige Sortierung.
- **Count()** — Erlaubt `?$count` : Gesamtanzahl mitzurückgeben.
- **SetMaxTop(100)** — Verhindert, dass jemand unbegrenzt viele Datensätze abrufen.

4.6 Generische Basisklasse: `ODataCrudController<T>` (DRY)

Ohne eine Basisklasse würden `AdressenController` und `CitiesController` denselben GET/POST/PATCH/DELETE-Code enthalten — eine klare Verletzung des DRY-Prinzips (Don't Repeat Yourself). Die Lösung ist eine generische abstrakte Basisklasse, die die gemeinsame CRUD-Logik einmalig implementiert. Unterklassen definieren nur noch, welches `DbSet` sie verwenden.

```
Controllers/ODataCrudController.cs
```

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.OData.Routing.Controllers;
using Microsoft.AspNetCore.OData.Query;
using Microsoft.AspNetCore.OData.Deltas;
using Microsoft.EntityFrameworkCore;
using AdressverwaltungApi.Data;

namespace AdressverwaltungApi.Controllers;

/// <summary>
/// Generische Basisklasse für OData-CRUD-Controller (DRY-Prinzip).
/// Unterklassen definieren nur: welches DbSet + welcher Anzeigename.
/// </summary>
public abstract class ODataCrudController

```

```

        [FromBody] Delta<TEntity> delta)
    {
        if (!ModelState.IsValid) return BadRequest(ModelState);
        var entity = await Entities.FindAsync(key);
        if (entity is null)
            return NotFound($"{EntityDisplayName} mit Id {key} wurde nicht gefunden.");
        delta.Patch(entity);
        if (!TryValidateModel(entity)) return BadRequest(ModelState);
        await _context.SaveChangesAsync();
        return Updated(entity);
    }

    // DELETE /odata/{EntitySet}(5)
    public virtual async Task<IActionResult> Delete([FromRoute] int key)
    {
        var entity = await Entities.FindAsync(key);
        if (entity is null)
            return NotFound($"{EntityDisplayName} mit Id {key} wurde nicht gefunden.");
        Entities.Remove(entity);
        await _context.SaveChangesAsync();
        return NoContent();
    }
}

```

Dank der Basisklasse werden die beiden Controller sehr kompakt:

```

Controllers/AdressenController.cs

```

```

public class AdressenController : ODataCrudController<Adresse>
{
    protected override DbSet<Adresse> Entities => _context.Adressen;
    protected override string EntityDisplayName => "Adresse";

    private readonly INotificationService _notificationService;

    public AdressenController(
        AdresseDbContext context,
        INotificationService notificationService,
        ILogger<AdressenController> logger) : base(context)
    {
        _notificationService = notificationService;
    }

    // POST überschrieben: Benachrichtigung nach dem Speichern auslösen
    public override async Task<IActionResult> Post([FromBody] Adresse adresse)
    {
        if (!ModelState.IsValid) return BadRequest(ModelState);
        Entities.Add(adresse);
        await _context.SaveChangesAsync();
        await _notificationService.NotifyNewAdresseAsync(adresse);
        return Created(adresse);
    }
}

// Controllers/CitiesController.cs (nur 7 Zeilen!)
public class CitiesController : ODataCrudController<City>
{
    protected override DbSet<City> Entities => _context.Cities;
    protected override string EntityDisplayName => "Stadt";
    public CitiesController(AdresseDbContext context) : base(context) { }
}

```

virtual und override — Polymorphismus

virtual in der Basisklasse bedeutet, dass Unterklassen die Methode überschreiben *können*. **override** in der Unterklasse ersetzt die Basisimplementierung. So kann **Post** im **AdressenController** angepasst werden (Benachrichtigung hinzufügen), während GET, PATCH und DELETE unverändert aus der Basisklasse kommen.

Konzept

Erklärung

ODataController

Basisklasse für OData-Controller. Stellt OData-spezifische Hilfsmethoden bereit (**Created()** , **Updated()**).

[EnableQuery]

Aktiviert OData-Abfrageparameter (**\$filter** , **\$orderby** , etc.) für diese Methode.

`Delta<T>` OData-Typ für Partial-Updates. Enthält nur die Felder, die im PATCH-Body gesendet wurden.

`AsNoTracking()` EF Core: Kein Change-Tracking bei GET — schneller und memory-effizienter für Leseanfragen.

`abstract` Die Klasse kann nicht direkt instanziiert werden — sie muss vererbt werden.

4.7 DbSeeder: Seed-Logik ausgelagert (SRP)

In einer früheren Version enthielt `Program.cs` die Logik für das Anlegen des Admin-Benutzers direkt — inklusive Magic Strings (Passwort hart codiert). Das verletzt das Single Responsibility Principle: `Program.cs` sollte nur für Konfiguration und Middleware zuständig sein. Die Lösung ist eine eigene statische Klasse `DbSeeder` :

```
DbSeeder.cs
```

```

using Microsoft.AspNetCore.Identity;
using AdressverwaltungApi.Data;
using AdressverwaltungApi.Models;

namespace AdressverwaltungApi;

/// <summary>
/// Befüllt die Datenbank mit initialen Daten (Admin-Benutzer).
/// Ausgelagert aus Program.cs gemäss SRP.
/// </summary>
public static class DbSeeder
{
    public static void Seed(
        AdresseDbContext db,
        IPasswordHasher<User> hasher,
        IConfiguration config)
    {
        if (db.Users.Any()) return; // Nur einmalig seeden

        var email = config["Seed:AdminEmail"] ?? "admin@example.com";
        var password = config["Seed:AdminPassword"] ?? "Test-1234!";

        var admin = new User
        {
            Email = email,
            DisplayName = "Admin",
        };
        admin.PasswordHash = hasher.HashPassword(admin, password);

        db.Users.Add(admin);
        db.SaveChanges();
    }
}

```

In `appsettings.json` kann der Standard-Admin konfiguriert werden:

`appsettings.json` (Erweiterung)

```

"Seed": {
  "AdminEmail": "admin@example.com",
  "AdminPassword": "Test-1234!"
}

```

Seed-Daten in Produktionsumgebungen

Die Seed-Zugangsdaten sollten in Produktionsumgebungen über Umgebungsvariablen gesetzt werden, nicht in `appsettings.json`. In Docker: `environment: - Seed__AdminEmail=admin@firma.ch` in der `docker-compose.yml`.

4.8 DTOs in eigenen Dateien (SRP)

DTOs (Data Transfer Objects) definieren, welche Daten eine API-Anfrage enthält. Früher standen `RegisterRequest`, `LoginRequest` und `SettingsUpdateRequest` direkt in den Controller-Dateien — das verletzt das SRP. Eigene Dateien im `Dtos/`-Ordner lösen dieses Problem:

Dtos/AuthDtos.cs

```
using System.ComponentModel.DataAnnotations;

namespace AdressverwaltungApi.Dtos;

public record RegisterRequest(
    [Required][EmailAddress] string Email,
    [Required][MinLength(8)] string Password,
    [Required][MaxLength(100)] string DisplayName
);

public record LoginRequest(
    [Required][EmailAddress] string Email,
    [Required] string Password
);
```

Dtos/SettingsDtos.cs

```
using System.ComponentModel.DataAnnotations;

namespace AdressverwaltungApi.Dtos;

public record SettingsUpdateRequest(
    [Required][EmailAddress][MaxLength(256)] string NotificationEmail
);
```

record vs. class — wann record?

C# Records sind unveränderlich (immutable) und haben automatisch `Equals`, `GetHashCode` und `ToString`. Für DTOs sind Records ideal, da ein Request-Body nach dem Empfang nie verändert werden sollte. Die verknüpfte `namespace AdressverwaltungApi.Dtos`-Deklaration sorgt für einen sauberen Import per `using AdressverwaltungApi.Dtos;` im Controller.

4.9 CORS konfigurieren

CORS (Cross-Origin Resource Sharing) ist ein Sicherheitsmechanismus des Browsers. Da unser Frontend auf Port `3000` läuft und das Backend auf Port `5000`, betrachtet der Browser dies als unterschiedliche Origins. Ohne CORS-Konfiguration blockiert der Browser alle Anfragen vom Frontend zum Backend.

Die CORS-Konfiguration haben wir bereits in `Program.cs` eingefügt:

Program.cs – CORS-Abschnitt (Auszug)

```
// In der Service-Konfiguration:
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowNextJs", policy =>
    {
        policy.WithOrigins("http://localhost:3000") // Next.js-URL
               .AllowAnyHeader()
               .AllowAnyMethod();
    });
});

// In der Middleware-Pipeline (VOR UseAuthorization):
app.UseCors("AllowNextJs");
```

Achtung: Reihenfolge der Middleware

`app.UseCors()` muss in der Middleware-Pipeline **vor** `app.UseAuthorization()` und `app.MapControllers()` stehen. Eine falsche Reihenfolge führt dazu, dass CORS-Header nicht gesetzt werden und der Browser alle Anfragen blockiert.

4.10 Backend testen mit REST Client

Starte das Backend:

bash

```
dotnet run
# Erwartet: Now listening on: http://localhost:5000
```

Erstelle die Datei `backend/api-tests.http` für die VS Code REST Client Extension. Diese Datei enthält HTTP-Anfragen, die du direkt in VS Code ausführen kannst (Klick auf *Send Request* über jeder Anfrage):

backend/api-tests.http

```

### Alle Adressen abrufen
GET http://localhost:5000/odata/Adressen
Accept: application/json

### Einzelne Adresse abrufen (Id=1)
GET http://localhost:5000/odata/Adressen(1)
Accept: application/json

### Adressen filtern (nur Bern)
GET http://localhost:5000/odata/Adressen?$filter=Ort eq 'Bern'
Accept: application/json

### Adressen alphabetisch sortieren
GET http://localhost:5000/odata/Adressen?$orderby=Name asc
Accept: application/json

### Neue Adresse erstellen (POST)
POST http://localhost:5000/odata/Adressen
Content-Type: application/json

{
  "Vorname": "Max",
  "Name": "Muster",
  "Strasse": "Dorfstrasse",
  "Strassennummer": "99",
  "Plz": "6340",
  "Ort": "Baar"
}

### Adresse aktualisieren (PATCH – nur geänderte Felder)
PATCH http://localhost:5000/odata/Adressen(1)
Content-Type: application/json

{
  "Ort": "Winterthur",
  "Plz": "8400"
}

### Adresse löschen (DELETE)
DELETE http://localhost:5000/odata/Adressen(6)
Accept: application/json

```

OData-Metadaten abrufen

Unter `GET http://localhost:5000/odata/$metadata` gibt das OData-Backend ein XML-Dokument zurück, das die komplette API-Struktur beschreibt (alle Entities, Felder, Typen). Tools wie Postman oder OData-fähige Clients können dieses Dokument lesen und die API automatisch verstehen.

Tipp: Swagger UI nutzen

Im Development-Modus steht auch die Swagger UI unter `http://localhost:5000/swagger` zur Verfügung. Hier kannst du alle Endpunkte interaktiv testen, ohne ein separates HTTP-Client-Tool zu benötigen.

Zusammenfassung Kapitel 4

- Das ASP.NET Core Web API-Projekt ist mit OData und EF Core konfiguriert.
- `Adresse.cs` (Model), `AdresseDbContext.cs` (Datenbankzugriff) und `AdressenController.cs` (Endpunkte) bilden das Backend.
- EF Core Migrations erzeugen die Datenbankstruktur auf Basis des C#-Modells.
- Alle fünf CRUD-Endpunkte (GET alle, GET eine, POST, PATCH, DELETE) sind implementiert.
- CORS erlaubt dem Next.js-Frontend auf Port 3000, auf das Backend zuzugreifen.

KAPITEL 5

5. Frontend: Next.js

Lernziele dieses Kapitels

- Du kannst ein Next.js-Projekt mit TypeScript, Tailwind CSS und App Router erstellen.
- Du verstehst die Struktur des Next.js App Routers und kannst Seiten und Layouts erstellen.
- Du kannst API-Calls mit der Fetch API gegen ein OData-Backend durchführen.
- Du kannst alle vier CRUD-Operationen im Frontend implementieren: Anzeigen, Erstellen, Bearbeiten und Löschen.

5.1 Neues Next.js-Projekt erstellen

Wechsle in das Frontend-Verzeichnis und erstelle ein neues Next.js-Projekt:

```
bash
```

```
cd adressverwaltung/frontend  
  
npx create-next-app@latest . --typescript --eslint --tailwind --app --no-src-dir --  
import-alias "@/*"
```

Der Befehl beantwortet alle Fragen automatisch. Die wichtigsten Optionen:

- `--typescript` — Aktiviert TypeScript für typsichere Entwicklung.
- `--tailwind` — Installiert und konfiguriert Tailwind CSS.
- `--app` — Verwendet den neuen App Router (empfohlen seit Next.js 13).
- `.` — Erstellt das Projekt im aktuellen Verzeichnis (nicht in einem neuen Unterordner).

Starte den Entwicklungsserver zum Testen:

```
bash
npm run dev
# Erwartet: Ready - started server on 0.0.0.0:3000
```

Öffne `http://localhost:3000` im Browser. Du siehst die Next.js-Startseite. Stoppe mit **Ctrl+C**.

Backend-URL konfigurieren

Erstelle die Datei `.env.local` im `frontend/`-Ordner für die Backend-URL. Diese Datei wird nie ins Repository eingchecked:

```
frontend/.env.local
NEXT_PUBLIC_API_URL=http://localhost:5000
```

5.2 API-Helper und TypeScript-Typen

TypeScript-Typdefinition: types/adresse.ts

Definiere zuerst den TypeScript-Typ für eine Adresse. Dieser Typ entspricht exakt dem C#-Model im Backend:

```
types/adresse.ts
```

```
// Repräsentiert eine vollständige Adresse (wie in der Datenbank)
export interface Adresse {
  id: number;
  vorname: string;
  name: string;
  strasse: string;
  strassennummer: string;
  plz: string;
  ort: string;
}

// Typ für das Erstellen einer neuen Adresse (ohne Id)
export type AdresseCreate = Omit<Adresse, 'id'>;

// Typ für das Aktualisieren (alle Felder optional ausser Id)
export type AdresseUpdate = Partial<AdresseCreate>;
```

API-Helper: lib/api.ts

Erstelle den Ordner `lib/` und die Datei `api.ts`. Diese Datei enthält alle Funktionen für die Kommunikation mit dem OData-Backend:

```
lib/api.ts
```

```

import { Adresse, AdresseCreate, AdresseUpdate } from '@types/adresse';

const BASE_URL = process.env.NEXT_PUBLIC_API_URL ?? 'http://localhost:5000';
const ODATA_URL = `${BASE_URL}/odata/Adressen`;

// OData gibt Daten in einem Wrapper-Objekt zurück: { value: [...] }
interface ODataResponse<T> {
  value: T[];
}

// =====
// GET /odata/Adressen - Alle Adressen abrufen
// =====
export async function getAlleAdressen(): Promise<Adresse[]> {
  const response = await fetch(ODATA_URL);

  if (!response.ok) {
    throw new Error(`Fehler beim Abrufen der Adressen: ${response.statusText}`);
  }

  const data: ODataResponse<Adresse> = await response.json();
  return data.value;
}

// =====
// GET /odata/Adressen(id) - Einzelne Adresse abrufen
// =====
export async function getAdresse(id: number): Promise<Adresse> {
  const response = await fetch(`${ODATA_URL}(${id})`);

  if (!response.ok) {
    throw new Error(`Adresse ${id} nicht gefunden: ${response.statusText}`);
  }

  return response.json();
}

// =====
// POST /odata/Adressen - Neue Adresse erstellen
// =====
export async function erstelleAdresse(adresse: AdresseCreate): Promise<Adresse> {
  const response = await fetch(ODATA_URL, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(adresse),
  });

  if (!response.ok) {
    throw new Error(`Fehler beim Erstellen: ${response.statusText}`);
  }
}

```

```

    return response.json();
  }

  // =====
  // PATCH /odata/Adressen(id) - Adresse aktualisieren
  // =====
  export async function aktualisiereAdresse(
    id: number,
    aenderungen: AdresseUpdate
  ): Promise<Adresse> {
    const response = await fetch(`${ODATA_URL}(${id})`, {
      method: 'PATCH',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(aenderungen),
    });

    if (!response.ok) {
      throw new Error(`Fehler beim Aktualisieren: ${response.statusText}`);
    }

    return response.json();
  }

  // =====
  // DELETE /odata/Adressen(id) - Adresse löschen
  // =====
  export async function loescheAdresse(id: number): Promise<void> {
    const response = await fetch(`${ODATA_URL}(${id})`, {
      method: 'DELETE',
    });

    if (!response.ok) {
      throw new Error(`Fehler beim Löschen: ${response.statusText}`);
    }
  }
}

```

Hilfsfunktion *normalizeAuditFields* (DRY, F-01)

In Version 2.0 wird die Normalisierung der sechs Audit-Felder (`createDate` , `createdBy` , `changeDate` , `changedBy` , `dateFrom` , `dateTo`) in eine gemeinsame Hilfsfunktion ausgelagert. Vorher wiederholten `normalizeAdresse` und `normalizeCity` denselben Code — eine Verletzung von DRY:

```
lib/api.ts (Ausschnitt – F-01: DRY)
```

```
// F-01: Gemeinsame Audit-Feld-Normalisierung (DRY-Prinzip)
// Vorher: 6 Zeilen in normalizeAdresse + 6 Zeilen in normalizeCity = Duplikat
// Nachher: einmal definiert, überall verwendbar
function normalizeAuditFields(raw: Record<string, unknown>) {
  return {
    createDate: String(raw['createDate'] ?? raw['CreateDate'] ?? ''),
    createdBy: String(raw['createdBy'] ?? raw['CreatedBy'] ?? ''),
    changeDate: (raw['changeDate'] ?? raw['ChangeDate'] ?? null) as string | null,
    changedBy: (raw['changedBy'] ?? raw['ChangedBy'] ?? null) as string | null,
    dateFrom: String(raw['dateFrom'] ?? raw['DateFrom'] ?? ''),
    dateTo: (raw['dateTo'] ?? raw['DateTo'] ?? null) as string | null,
  };
}

function normalizeAdresse(raw: Record<string, unknown>): Adresse {
  return {
    id: Number(raw['id'] ?? raw['Id']),
    vorname: String(raw['vorname'] ?? raw['Vorname'] ?? ''),
    name: String(raw['name'] ?? raw['Name'] ?? ''),
    strasse: String(raw['strasse'] ?? raw['Strasse'] ?? ''),
    strassennummer: String(raw['strassennummer'] ?? raw['Strassennummer'] ?? ''),
    plz: String(raw['plz'] ?? raw['Plz'] ?? ''),
    ort: String(raw['ort'] ?? raw['Ort'] ?? ''),
    ...normalizeAuditFields(raw), // ← spread-Operator: Audit-Felder einbinden
  };
}
```

5.3 Adressliste: app/page.tsx (GET, F-03, F-06)

In Version 2.0 wurden zwei Clean-Code-Befunde in `page.tsx` umgesetzt: `confirm()` / `alert()` wurden durch eine eigene `ConfirmDialog`-Komponente ersetzt (F-03), und `ladeDaten` wird mit `useCallback` stabilisiert (F-06). Erstelle die Datei `app/page.tsx`:

```
app/page.tsx
```

```

'use client';

import { useEffect, useState, useCallback } from 'react';
import Link from 'next/link';
import { Adresse } from '@types/adresse';
import { getAlleAdressen, loescheAdresse } from '@lib/api';
import ConfirmDialog from '@components/ConfirmDialog';

export default function AdressenListePage() {
  const [adressen, setAdressen] = useState<Adresse[]>([]);
  const [ladevorgang, setLadevorgang] = useState(true);
  const [fehler, setFehler] = useState<string | null>(null);

  // F-03: State für Bestätigungsdialog (ersetzt nativen confirm())
  const [zuLoeschendId, setZuLoeschendId] = useState<number | null>(null);
  const [loeschFehler, setLoeschFehler] = useState<string | null>(null);

  // F-06: useCallback – stabile Referenz, kein Rebuild bei jedem Render
  const ladeDaten = useCallback(async () => {
    try {
      setLadevorgang(true);
      setFehler(null);
      const daten = await getAlleAdressen();
      setAdressen(daten);
    } catch (err) {
      setFehler('Fehler beim Laden. Läuft das Backend? (http://localhost:5000)');
      console.error(err);
    } finally {
      setLadevorgang(false);
    }
  }, []);

  useEffect(() => { ladeDaten(); }, [ladeDaten]);

  // Löschen: Dialog öffnen
  function loeschenBestaetigen(id: number) {
    setLoeschFehler(null);
    setZuLoeschendId(id);
  }

  // Löschen: nach Bestätigung ausführen
  async function handleLoeschenBestaetigt() {
    if (zuLoeschendId === null) return;
    const id = zuLoeschendId;
    setZuLoeschendId(null);
    try {
      await loescheAdresse(id);
      setAdressen(prev => prev.filter(a => a.id !== id));
    } catch (err) {
      setLoeschFehler('Fehler beim Löschen. Bitte versuche es erneut.');
```



```

        <td className="px-4 py-3">{adresse.strasse}</td>
        <td className="px-4 py-3">{adresse.strassennummer}</td>
        <td className="px-4 py-3">{adresse.plz}</td>
        <td className="px-4 py-3">{adresse.ort}</td>
        <td className="px-4 py-3 flex gap-2">
          <Link href={`/adressen/${adresse.id}/bearbeiten`}
            className="text-blue-600 hover:underline">Bearbeiten</Link>
          <button onClick={() => loeschenBestaetigen(adresse.id)}
            className="text-red-500 hover:underline">Löschen</button>
        </td>
      </tr>
    ))
  </tbody>
</table>
</div>
<p className="text-gray-400 text-sm mt-4">{adressen.length} Adresse(n)
gefunden</p>
</main>
);
}

```

5.4 ConfirmDialog: Eigener Bestätigungsdialog (F-03)

`window.confirm()` ist ein nativer Browser-Dialog — er lässt sich nicht gestalten, funktioniert in iFrames nicht und hat schlechte Accessibility (Barrierefreiheit). Die Lösung: eigene React-Komponente `ConfirmDialog`.

Aspekt	<code>window.confirm()</code>	<code>ConfirmDialog</code> -Komponente
Gestaltbar	Nein	Ja (Tailwind CSS)
Accessibility	Schlecht	Gut (ARIA-Attribute)
iFrame-kompatibel	Nein	Ja
State-basiert	Nein	Ja (React-Paradigma)
Testbar	Schwer	Einfach

```
components/ConfirmDialog.tsx
```

```

'use client';

interface Props {
  offen: boolean;
  meldung: string;
  onBestaetigen: () => void;
  onAbbrechen: () => void;
}

export default function ConfirmDialog({ offen, meldung, onBestaetigen, onAbbrechen }:
Props) {
  if (!offen) return null;

  return (
    <div
      className="fixed inset-0 z-50 flex items-center justify-center bg-black/40"
      role="dialog"
      aria-modal="true"
      aria-labelledby="dialog-meldung"
    >
      <div className="bg-white rounded-xl shadow-xl p-6 max-w-sm w-full mx-4">
        <p id="dialog-meldung" className="text-gray-800 mb-6 text-base">
          {meldung}
        </p>
        <div className="flex justify-end gap-3">
          <button onClick={onAbbrechen}
            className="px-4 py-2 rounded-lg border border-gray-300 text-gray-700
            hover:bg-gray-50">
            Abbrechen
          </button>
          <button onClick={onBestaetigen}
            className="px-4 py-2 rounded-lg bg-red-600 text-white hover:bg-red-700">
            Löschen
          </button>
        </div>
      </div>
    </div>
  );
}

```

ARIA-Attribute — was sind das?

`role="dialog"` und `aria-modal="true"` sind Accessibility-Attribute (ARIA). Sie teilen Screen-Readern mit, dass dies ein modaler Dialog ist. `aria-labelledby="dialog-meldung"` verknüpft den Dialog-Titel mit dem Dialog-Element und verbessert so die Zugänglichkeit für Menschen mit Sehbehinderungen.

5.5 AdresseForm: Controlled Inputs (F-02, F-05)

In Version 2.0 wurden alle Formularfelder auf **Controlled Inputs** umgestellt (F-02). Ausserdem wurde der Tippfehler `suchelaeuft` zu `sucheLaeuft` korrigiert (F-05).

Uncontrolled Input (alt): `<input defaultValue={wert}>` — React kontrolliert den Wert nicht; Auslesen erfordert `form.elements.namedItem()`.

Controlled Input (neu): `<input value={wert} onChange={e => setWert(e.target.value)}>` — React kontrolliert den Wert via State; der aktuelle Wert steht immer direkt in der State-Variable.

```
components/AdresseForm.tsx (Ausschnitt – State-Deklaration)
```

```

'use client';

import { useState } from 'react';
import { AdresseCreate } from '@/types/adresse';

interface Props {
  initialWerte?: Partial<AdresseCreate>;
  onSubmit: (daten: AdresseCreate) => Promise<void>;
  submitLabel: string;
}

export default function AdresseForm({ initialWerte = {}, onSubmit, submitLabel }: Props)
{
  // F-02: Alle Felder als Controlled Inputs (einheitlich via useState)
  const [vorname, setVorname] = useState(initialWerte.vorname ?? '');
  const [name, setName] = useState(initialWerte.name ?? '');
  const [strasse, setStrasse] = useState(initialWerte.strasse ?? '');
  const [strassennummer, setStrassennummer] = useState(initialWerte.strassennummer ?? '');
  const [plz, setPlz] = useState(initialWerte.plz ?? '');
  const [ort, setOrt] = useState(initialWerte.ort ?? '');

  // F-05: Korrekter camelCase-Name (Tippfehler 'suchelaeuft' behoben)
  const [sucheLaeuft, setSucheLaeuft] = useState(false);

  // handleSubmit liest direkt aus State (kein form.elements.namedItem mehr nötig)
  async function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault();
    const daten: AdresseCreate = {
      vorname: vorname.trim(),
      name: name.trim(),
      strasse: strasse.trim(),
      strassennummer: strassennummer.trim(),
      plz: plz.trim(),
      ort: ort.trim(),
    };
    await onSubmit(daten);
  }

  const inputClass =
    'w-full border border-gray-300 rounded px-3 py-2 focus:outline-none focus:ring-2 focus:ring-blue-500';
  const labelClass = 'block text-sm font-medium text-gray-700 mb-1';

  return (
    <form onSubmit={handleSubmit} className="space-y-4">
      <div className="grid grid-cols-2 gap-4">
        <div>

```

```

        <label className={labelClass}>Vorname *</label>
        <input required value={vorname}
          onChange={e => setVorname(e.target.value)} className={inputClass} />
      </div>
      <div>
        <label className={labelClass}>Nachname *</label>
        <input required value={name}
          onChange={e => setName(e.target.value)} className={inputClass} />
      </div>
    </div>
    { /* PLZ + Ort: zeigt suchelaeuft-Spinner wenn PLZ-Autocomplete aktiv */ }
    { sucheLaeuft && <p className="text-xs text-gray-400">Suche läuft...</p> }
    { /* ... weitere Felder analog */ }
    <button type="submit"
      className="bg-blue-600 text-white px-6 py-2 rounded hover:bg-blue-700 font-
medium">
      {submitLabel}
    </button>
  </form>
);
}

```

5.6 TypeScript: NextAuth Session-Augmentation (F-04)

Das TypeScript-Typsensystem kennt `session.user.id` standardmässig nicht, weil NextAuth die Session-Typen nicht mit einem `id`-Feld deklariert. Früher wurde ein unsicherer Type-Cast verwendet: `(session.user as { id?: string }).id`. Die Lösung heisst **Module Augmentation**:

types/next-auth.d.ts

```

import 'next-auth';

declare module 'next-auth' {
  interface Session {
    user: {
      id: string;
      name?: string | null;
      email?: string | null;
      image?: string | null;
    };
  }
}

declare module 'next-auth/jwt' {
  interface JWT {
    id?: string;
  }
}

```

Was ist Module Augmentation?

`declare module` öffnet ein bestehendes TypeScript-Modul und erweitert seine Typen. Das ist der TypeScript-Standardweg, um fremde Typen zu erweitern, ohne den Quellcode der Bibliothek zu ändern. Die Datei muss in `tsconfig.json` unter `include` liegen (standardmässig ist `types/` enthalten).

Zusammenfassung Kapitel 5

- Das Next.js-Projekt ist mit TypeScript, Tailwind CSS und App Router konfiguriert.
- `normalizeAuditFields` als gemeinsame Hilfsfunktion vermeidet Duplikate (DRY, F-01).
- `app/page.tsx` verwendet `ConfirmDialog` statt `confirm()` (F-03) und `useCallback` für `ladeDaten` (F-06).
- `AdresseForm` verwendet einheitlich Controlled Inputs (F-02) und den korrekten Namen `sucheLaeuft` (F-05).
- `types/next-auth.d.ts` erweitert den Session-Typ via Module Augmentation (F-04).

KAPITEL 6

6. Zusammenspiel testen

Lernziele dieses Kapitels

- Du kannst Backend und Frontend gleichzeitig starten und die Kommunikation zwischen ihnen testen.
- Du kannst alle CRUD-Operationen manuell im Browser durchführen und das Ergebnis überprüfen.
- Du kennst die häufigsten Fehler (CORS, Connection String, OData-Routing) und weisst, wie du sie behebst.

6.1 Backend und Frontend starten

Für den manuellen Test müssen beide Server gleichzeitig laufen. Öffne zwei separate Terminals in VS Code (**Ctrl+Shift+5** für ein geteiltes Terminal):

Terminal 1 – Backend starten:

```
bash - Terminal 1
```

```
cd adressverwaltung/backend/AdressverwaltungApi
dotnet run
# Erwartet: Now listening on: http://localhost:5000
```

Terminal 2 – Frontend starten:

```
bash - Terminal 2

cd adressverwaltung/frontend
npm run dev
# Erwartet: Ready - started server on 0.0.0.0:3000
```

Öffne nun `http://localhost:3000` im Browser. Du siehst die Adressliste mit den zuvor eingefügten Testdaten.

Manuelle CRUD-Tests

Aktion	Vorgehen im Browser	Erwartetes Ergebnis
Alle anzeigen	Startseite öffnen	Tabelle mit 5 Testadressen
Neue erstellen	«+ Neue Adresse» klicken, Formular ausfüllen, speichern	Adresse erscheint in der Liste
Bearbeiten	«Bearbeiten» bei einer Adresse klicken, Feld ändern, speichern	Aktualisierter Wert in der Liste
Löschen	«Löschen» klicken, Bestätigung mit OK	Adresse verschwindet aus der Liste

Browser Developer Tools nutzen

Öffne die Browser-Entwicklertools mit **F12** und wechsle zum Tab **Network**. Hier siehst du alle HTTP-Anfragen, die das Frontend an das Backend sendet. Wichtige Punkte beim Beobachten:

- Beim Laden der Startseite: `GET /odata/Adressen` — Status 200
- Beim Erstellen: `POST /odata/Adressen` — Status 201
- Beim Bearbeiten: `PATCH /odata/Adressen(id)` — Status 200
- Beim Löschen: `DELETE /odata/Adressen(id)` — Status 204

Tipp: console.log für Debugging

Füge temporär `console.log(daten)` nach dem API-Call in `lib/api.ts` hinzu, um die Antwort des Backends im Browser-Tab «Console» zu sehen. Das hilft, die JSON-Struktur der OData-Antwort zu verstehen.

6.2 Häufige Fehler und Lösungen

Fehlermeldung	Ursache	Lösung
<code>CORS error</code> in der Browser-Konsole	CORS nicht oder falsch konfiguriert, oder <code>UseCors()</code> steht an der falschen Stelle in der Middleware-Pipeline	Sicherstellen, dass <code>app.UseCors("AllowNextJs")</code> in <code>Program.cs</code> vor <code>app.UseAuthorization()</code> steht. URL in der Policy (<code>http://localhost:3000</code>) prüfen.
<code>Failed to connect to database</code> beim Starten des Backends	PostgreSQL läuft nicht, oder Connection String ist falsch	PostgreSQL-Dienst starten (<code>sudo systemctl start postgresql</code>). Connection String in <code>appsettings.json</code> prüfen: Host, Port, Datenbankname, Benutzername, Passwort.
<code>404 Not Found</code> bei <code>/odata/Adressen</code>	OData-Route falsch konfiguriert, oder Controller nicht gefunden	In <code>Program.cs</code> prüfen: <code>.AddRouteComponents("odata", ...)</code> . Controller-Klasse muss von <code>ODataController</code> erben (nicht <code>ControllerBase</code>).
<code>relation "Adressen" does not exist</code>	EF Core Migration nicht ausgeführt, oder Tabellenname stimmt nicht überein	<code>dotnet ef database update</code> ausführen. In <code>AdresseDbContext.cs</code> prüfen: <code>entity.ToTable("Adressen")</code> mit Grossbuchstaben.
Frontend zeigt leere Liste, kein Fehler	OData gibt <code>{ value: [] }</code> zurück – Datenbank ist leer	Testdaten per SQL einfügen (Kapitel 3) oder über das Frontend eine neue Adresse erstellen.
<code>dotnet-ef: command not found</code>	EF Core Tool nicht installiert oder nicht im PATH	<code>dotnet tool install --global dotnet-ef</code> ausführen. Dann PATH-Variable aktualisieren (siehe Kapitel 2).

Zusammenfassung Kapitel 6

- Backend (Port 5000) und Frontend (Port 3000) laufen gleichzeitig in zwei Terminals.

- Alle vier CRUD-Operationen können manuell im Browser getestet werden.
- Die Browser-Entwicklertools (Network-Tab) zeigen alle HTTP-Anfragen und -Antworten.
- Die häufigsten Fehler sind CORS-Probleme, falsche Connection Strings und fehlende Migrationen.

KAPITEL 7

7. Dockerisierung

Lernziele dieses Kapitels

- Du verstehst, was Docker ist und warum es für die Softwareentwicklung nützlich ist.
- Du kannst ein Dockerfile für ein ASP.NET Core-Projekt und ein Next.js-Projekt schreiben.
- Du kannst eine `docker-compose.yml`-Datei mit drei Services konfigurieren.
- Du kannst die gesamte Applikation mit einem einzigen Befehl starten und stoppen.

7.1 Was ist Docker?

Docker ist eine Plattform, mit der Applikationen in sogenannten *Containern* verpackt und ausgeführt werden können. Ein Container enthält die Applikation zusammen mit allen Abhängigkeiten (Laufzeitumgebung, Bibliotheken, Konfiguration) – alles was nötig ist, um die Applikation zu starten.

Der wichtigste Vorteil von Docker: **«Works on my machine»-Probleme gehören der Vergangenheit an.** Ein Container läuft auf jedem System gleich – auf deinem Entwicklungsrechner, auf dem Testserver und in der Produktionsumgebung.

Wichtige Docker-Begriffe

Begriff	Erklärung	Analogie
Image	Unveränderliche Vorlage für einen Container. Wird aus einem Dockerfile gebaut.	ISO-Datei / Installationsdatenträger
Container	Laufende Instanz eines Images. Kann gestartet, gestoppt und gelöscht werden.	Installierte Applikation
Dockerfile	Textdatei mit Anweisungen zum Bauen eines Images.	Kochrezept
Volume	Persistenter Speicher ausserhalb des Containers. Bleibt erhalten, wenn der Container gelöscht wird.	Externer USB-Stick

docker-compose	Tool zum Starten mehrerer Container als ein Verbund.	Orchesterleiter
Registry	Speicherort für Docker Images (z.B. Docker Hub).	App Store

Multi-Stage Builds

Unsere Dockerfiles verwenden **Multi-Stage Builds**: In der ersten Stage wird der Code kompiliert (Build-Stage), in der zweiten Stage wird nur das fertige Ergebnis in ein schlankes Runtime-Image kopiert. Das Endresultat ist deutlich kleiner als ein Image, das alle Build-Tools enthält.

7.2 Dockerfiles erstellen

Backend-Dockerfile

Erstelle die Datei `backend/AdressverwaltungApi/Dockerfile` :

```
backend/AdressverwaltungApi/Dockerfile
```

```
# =====
# Stage 1: Build (SDK enthält Compiler und dotnet-Tools)
# =====
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /app

# Zuerst nur die Projektdaten kopieren (für Docker Layer Caching)
COPY *.csproj ./
RUN dotnet restore

# Dann den restlichen Code kopieren und builden
COPY . ./
RUN dotnet publish -c Release -o /app/publish

# =====
# Stage 2: Runtime (nur die ASP.NET Runtime, kein SDK)
# =====
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app

# Nur das kompilierte Ergebnis aus Stage 1 kopieren
COPY --from=build /app/publish .

# Port für die API (HTTP)
EXPOSE 8080

# Umgebungsvariable: ASP.NET Core soll auf Port 8080 lauschen
ENV ASPNETCORE_URLS=http://+:8080

# Startbefehl
ENTRYPOINT ["dotnet", "AdressverwaltungApi.dll"]
```

Frontend-Dockerfile

Erstelle die Datei `frontend/Dockerfile`:

```
frontend/Dockerfile
```

```

# =====
# Stage 1: Dependencies installieren
# =====
FROM node:20-alpine AS deps
WORKDIR /app

# package.json und package-lock.json kopieren
COPY package.json package-lock.json* ./
RUN npm ci --only=production

# =====
# Stage 2: Build
# =====
FROM node:20-alpine AS builder
WORKDIR /app

# Alle Dependencies (inkl. devDependencies für den Build)
COPY package.json package-lock.json* ./
RUN npm ci

# Quellcode kopieren
COPY . .

# Umgebungsvariable für den Build setzen
# (API-URL zeigt auf den Docker-internen Service-Namen "backend")
ENV NEXT_PUBLIC_API_URL=http://backend:8080

# Next.js Build
RUN npm run build

# =====
# Stage 3: Runtime (minimales Image)
# =====
FROM node:20-alpine AS runtime
WORKDIR /app

ENV NODE_ENV=production
ENV PORT=3000

# Nur die nötigen Dateien aus Stage 2 kopieren
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/public ./public

EXPOSE 3000

CMD ["node", "server.js"]

```

Achtung: Next.js Standalone-Output aktivieren

Das Frontend-Dockerfile nutzt den `standalone` -Output von Next.js. Füge dazu in `frontend/next.config.ts` folgende Konfiguration hinzu:

```
frontend/next.config.ts

import type { NextConfig } from 'next';

const nextConfig: NextConfig = {
  output: 'standalone',
};

export default nextConfig;
```

.dockerignore-Dateien erstellen

Um unnötige Dateien aus dem Docker-Build auszuschliessen und den Build zu beschleunigen:

```
backend/AdressverwaltungApi/.dockerignore
```

```
bin/
obj/
*.user
.vs/
appsettings.Development.json
```

```
frontend/.dockerignore
```

```
node_modules/
.next/
.env.local
.git/
README.md
```

7.3 docker-compose.yml konfigurieren

Erstelle die Datei `adressverwaltung/docker-compose.yml`. Diese Datei definiert alle drei Services und wie sie miteinander kommunizieren:

```
docker-compose.yml
```

```

version: '3.9'

services:

# =====
# Service 1: PostgreSQL Datenbank
# =====
db:
  image: postgres:16-alpine
  container_name: adressverwaltung-db
  restart: unless-stopped
  environment:
    POSTGRES_DB: adressverwaltung
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: deinPasswort123
  ports:
    - "5432:5432"          # Lokaler Zugriff mit psql/pgAdmin weiterhin möglich
  volumes:
    - postgres-data:/var/lib/postgresql/data # Daten bleiben persistent

# =====
# Service 2: ASP.NET Core Backend
# =====
backend:
  build:
    context: ./backend/AdressverwaltungApi
    dockerfile: Dockerfile
  container_name: adressverwaltung-backend
  restart: unless-stopped
  ports:
    - "5000:8080"          # Externer Port 5000 -> Interner Port 8080
  environment:
    # Connection String überschreibt appsettings.json
    # "db" ist der Service-Name des Datenbankcontainers
    ConnectionStrings__DefaultConnection: >-
      Host=db;Port=5432;Database=adressverwaltung;
      Username=postgres;Password=deinPasswort123
    ASPNETCORE_ENVIRONMENT: Production
  depends_on:
    - db                  # Backend startet erst, wenn db-Container bereit ist

# =====
# Service 3: Next.js Frontend
# =====
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: adressverwaltung-frontend
  restart: unless-stopped
  ports:
    - "3000:3000"          # Externer Port 3000 -> Interner Port 3000
  depends_on:

```

```

- backend          # Frontend startet nach Backend

volumes:
  postgres-data:    # Named Volume für Datenbankpersistenz

```

Interne Docker-Netzwerkcommunication

Docker Compose erstellt automatisch ein internes Netzwerk für alle Services. Innerhalb dieses Netzwerks kann jeder Service die anderen über ihren **Service-Namen** als Hostname ansprechen. Das Backend nutzt `Host=db` statt `Host=localhost`, weil `db` der Service-Name des PostgreSQL-Containers ist.

7.4 Container ausführen und testen

Alle Befehle werden im Hauptverzeichnis `adressverwaltung/` ausgeführt:

```

bash - im Ordner adressverwaltung/

# Images bauen und alle Container starten
docker-compose up --build

# Im Hintergrund starten (ohne Logs)
docker-compose up --build -d

# Logs eines bestimmten Services anzeigen
docker-compose logs backend
docker-compose logs -f frontend    # -f = follow (live)

# Status aller Container anzeigen
docker-compose ps

# Alle Container stoppen (Daten bleiben erhalten)
docker-compose down

# Alle Container stoppen UND Volumes löschen (Datenverlust!)
docker-compose down -v

```

Nach dem Start ist die Applikation unter folgenden URLs erreichbar:

Service	URL	Zugriff von
Frontend (Next.js)	<code>http://localhost:3000</code>	Browser
Backend (OData API)	<code>http://localhost:5000/odata/Adressen</code>	Browser / REST Client
Datenbank (PostgreSQL)	<code>localhost:5432</code>	psql / pgAdmin

Achtung: CORS im Docker-Modus

Im Docker-Modus kommuniziert das Frontend intern mit dem Backend über `http://backend:8080` (Service-Name). Die CORS-Policy im Backend muss für den produktiven Docker-Betrieb entsprechend angepasst werden – entweder die Origin-URL via Umgebungsvariable konfigurieren, oder `AllowAnyOrigin()` nur für Entwicklungszwecke erlauben.

Tipp: Datenbank-Initialisierung beim ersten Start

Beim ersten Start des Docker-Stacks existiert die Tabelle "Adressen" noch nicht. Du hast zwei Möglichkeiten: (1) EF Core Migrations beim Backend-Start automatisch anwenden lassen (in `Program.cs` vor `app.Run()` : `using var scope = app.Services.CreateScope(); scope.ServiceProvider.GetRequiredService<AdresseDbContext>().Database.Migrate();`), oder (2) nach dem Start manuell `docker exec adressverwaltung-backend dotnet-ef database update` ausführen.

Zusammenfassung Kapitel 7

- Docker verpackt Applikationen mit allen Abhängigkeiten in portable Container.
- Multi-Stage Builds trennen Build-Umgebung und Runtime, was kleinere Images ergibt.
- Die `docker-compose.yml` definiert drei Services: `db` (PostgreSQL), `backend` (ASP.NET Core) und `frontend` (Next.js).
- Mit `docker-compose up --build` startet die gesamte Applikation mit einem einzigen Befehl.
- Named Volumes sichern die Datenbankdaten auch nach einem Neustart der Container.

KAPITEL 8

8. Stadt-Entität & PLZ-Wertehilfe

Lernziele dieses Kapitels

- Du kannst eine neue Datenbank-Entität (`City`) mit OData-Anbindung erstellen.
- Du verstehst, wie OData-Filter-Queries (`$filter=startswith(...)`) funktionieren.
- Du implementierst eine Autocomplete-Wertehilfe im Frontend mit Debouncing.
- Du weißt, wie eine neue Seite mit Tabelle, Formular und Routing in Next.js aufgebaut wird.

8.1 City-Model & Migration

Wir fügen eine neue Entität `City` hinzu, die als PLZ-Verzeichnis dient. Eine Stadt hat eine eindeutige Postleitzahl und einen Ortsnamen. Das Model wird im Ordner `Models/` des Backends angelegt.

Models/City.cs

```
using System.ComponentModel.DataAnnotations;

namespace AdressverwaltungApi.Models;

public class City
{
    public int Id { get; set; }

    [Required, MaxLength(10)]
    public string PostalCode { get; set; } = string.Empty;

    [Required, MaxLength(100)]
    public string CityName { get; set; } = string.Empty;
}
```

Damit EF Core die Tabelle anlegen kann, erweitern wir den `AdresseDbContext` um ein `DbSet<City>` und konfigurieren einen Datenbank-Index auf `PostalCode`. Ein Index macht Suchabfragen nach PLZ deutlich schneller.

Data/AdresseDbContext.cs (Ausschnitt)

```
public DbSet<City> Cities { get; set; }

// In OnModelCreating:
modelBuilder.Entity<City>()
    .HasIndex(c => c.PostalCode)
    .IsUnique()
    .HasDatabaseName("IX_Cities_PostalCode");
```

Anschliessend wird eine neue Migration erstellt und auf die Datenbank angewendet:

Terminal

```
cd backend/AdressverwaltungApi
dotnet ef migrations add AddCitiesUsersSettings
dotnet ef database update
```

Was macht ein Datenbank-Index?

Ohne Index muss die Datenbank bei jeder PLZ-Suche alle Zeilen der Tabelle durchlesen (Full Table Scan). Mit einem Index legt PostgreSQL intern eine sortierte Struktur an, ähnlich einem Stichwortverzeichnis in einem Buch – die Suche wird dadurch um ein Vielfaches schneller. Bei einer PLZ-Wertehilfe, die bei jeder Tastatureingabe eine Abfrage sendet, ist das besonders wichtig.

8.2 CitiesController (OData CRUD)

Der Controller stellt alle vier CRUD-Operationen als OData-Endpunkte bereit. Besonders wichtig ist, dass wir `AsNoTracking()` für lesende Abfragen verwenden – das verbessert die Performance, weil EF Core keine Änderungsverfolgung für diese Objekte startet.

Controllers/CitiesController.cs (Ausschnitt)

```
[ODataRouteComponent]
[ApiController]
[Route("odata")]
public class CitiesController : ODataController
{
    private readonly AdresseDbContext _db;

    public CitiesController(AdresseDbContext db) => _db = db;

    // GET /odata/Cities (mit OData-Filterung, Sortierung, Paginierung)
    [EnableQuery]
    public IQueryable<City> GetCities() =>
        _db.Cities.AsNoTracking();

    // GET /odata/Cities(5)
    [EnableQuery]
    public async Task<IActionResult> GetCity([FromRoute] int key) { ... }

    // POST /odata/Cities
    public async Task<IActionResult> PostCity([FromBody] City city) { ... }

    // PATCH /odata/Cities(5)
    public async Task<IActionResult> PatchCity([FromRoute] int key,
        [FromBody] Delta<City> delta) { ... }

    // DELETE /odata/Cities(5)
    public async Task<IActionResult> DeleteCity([FromRoute] int key) { ... }
}
```

In `Program.cs` muss die Entität dem OData-Modell bekanntgemacht werden:

Program.cs (Ausschnitt)

```
var builder = ODataConventionModelBuilder();
builder.EntitySet<Adresse>("Adressen");
builder.EntitySet<City>("Cities"); // NEU
```

Damit ist der Endpunkt `GET /odata/Cities?$filter=startswith(postalCode,'80')&$top=10` einsatzbereit. OData übersetzt diesen Query-Parameter automatisch in einen effizienten SQL-`LIKE`-Ausdruck.

8.3 Frontend: Typen & API-Funktionen

Im Frontend definieren wir TypeScript-Typen für City und ergänzen die API-Helfer-Datei um alle City-spezifischen Funktionen.

types/city.ts

```
export interface City {
  id: number;
  postalCode: string;
  cityName: string;
}

export type CityCreate = Omit<City, 'id'>;
export type CityUpdate = Partial<CityCreate>;
```

Die Suchfunktion für die PLZ-Wertehilfe verwendet einen OData-`$filter`-Parameter. Wichtig: Der Wert muss mit `encodeURIComponent` URL-sicher kodiert werden, da der Filter Apostrophe und Klammern enthält.

lib/api.ts (Ausschnitt)

```
// Suche nach PLZ-Präfix (für die Wertheilfe im Adressformular)
export async function sucheStaedteNachPlz(plzPrefix: string): Promise<City[]> {
  if (!plzPrefix || plzPrefix.length < 2) return [];

  const filter = encodeURIComponent(`startswith(postalCode, '${plzPrefix}')`);
  const response = await fetch(
    `${ODATA_URL}/Cities?$filter=${filter}&$top=10&$orderby=postalCode`,
    { cache: 'no-store' }
  );

  if (!response.ok) return [];
  const data = await response.json() as { value: Record<string, unknown>[] };
  return data.value.map(normalizeCity);
}
```

8.4 CityForm & Städte-Seiten

Analog zu `AdresseForm` erstellen wir eine `CityForm` -Komponente sowie drei Seiten:

Datei	Route	Funktion
<code>app/cities/page.tsx</code>	<code>/cities</code>	Tabelle aller Städte (Server Component)
<code>app/cities/neu/page.tsx</code>	<code>/cities/neu</code>	Neue Stadt anlegen (Client Component)
<code>app/cities/[id]/bearbeiten/page.tsx</code>	<code>/cities/:id/bearbeiten</code>	Stadt bearbeiten (Client Component)

Die Listenseite ist als **Server Component** implementiert: Sie lädt die Daten direkt beim Rendern auf dem Server, ohne einen JavaScript-`useEffect` -Hook. Das macht sie schneller und einfacher – sie braucht keinen Ladezustand im Client. Die «Löschen»-Schaltfläche wird als separate Client-Komponente (`DeleteButton.tsx`) ausgelagert, da sie einen Event-Handler benötigt.

Server Components vs. Client Components in Next.js 14

In Next.js 14 (App Router) ist jede Komponente standardmässig eine **Server Component**: Sie wird nur auf dem Server ausgeführt, hat keinen Zugriff auf Browser-APIs und kann `async/await` direkt verwenden. Eine Komponente mit `'use client'` am Dateianfang läuft im Browser und hat Zugriff auf State, Event-Handler und Browser-APIs. Die Faustregel: Muss die Komponente auf Benutzereingaben reagieren? → Client. Reicht reines Anzeigen von Daten? → Server.

8.5 PLZ-Autocomplete im Adressformular

Die PLZ-Wertehilfe ist ein klassisches Autocomplete-Pattern: Während der Benutzer tippt, werden passende Städte aus der API geladen und als Dropdown angezeigt. Zwei wichtige Konzepte kommen hier zum Einsatz:

Debouncing: Damit nicht bei jedem Tastendruck eine HTTP-Anfrage gesendet wird, wartet der Code 300 ms ab, bevor er die Suche auslöst. Tippt der Benutzer innerhalb dieser Zeit weiter, wird der Timer zurückgesetzt.

```
components/AdresseForm.tsx (PLZ-Wertehilfe, Ausschnitt)

// Debounced PLZ-Suche: API erst nach 300 ms aufrufen
useEffect(() => {
  if (plz.length < 2) { setVorschlaege([]); return; }

  const timer = setTimeout(async () => {
    const treffer = await sucheStaedteNachPlz(plz);
    setVorschlaege(treffer);
    setZeigeDropdown(treffer.length > 0);
  }, 300);

  return () => clearTimeout(timer); // Cleanup: Timer abbrechen
}, [plz]);

// Vorschlag auswählen → PLZ + Ort automatisch ausfüllen
const waehleVorschlag = (city: City) => {
  setPlz(city.postalCode);
  setOrt(city.cityName);
  setZeigeDropdown(false);
};
```

Klick-ausserhalb-Erkennung: Das Dropdown soll sich schliessen, wenn der Benutzer irgendwo ausserhalb des Feldes klickt. Dafür wird ein `mousedown`-Event-Listener auf `document` registriert, der prüft, ob der Klick innerhalb des Dropdown-Containers lag. Die Listener-Bereinigung im Cleanup der `useEffect`-Funktion verhindert Memory Leaks.

Zusammenfassung Kapitel 8

- Eine neue Entität wird im Backend als C#-Klasse (Model), DbSet im DbContext und OData-Eintrag in Program.cs definiert.
- EF Core-Migrationen halten Datenbankschema und C#-Code synchron.
- OData-Filterfunktionen wie `startswith()` werden automatisch in SQL übersetzt.
- Debouncing reduziert unnötige API-Aufrufe und verbessert die Benutzererfahrung.

- Server Components in Next.js 14 können Daten direkt beim Rendern laden – ohne `useEffect` .

KAPITEL 9

9. Benutzerverwaltung & Login

Lernziele dieses Kapitels

- Du kannst Benutzer mit E-Mail und Passwort im Backend registrieren und authentifizieren.
- Du verstehst, warum Passwörter niemals im Klartext gespeichert werden dürfen.
- Du richtest NextAuth.js mit Credentials- und Google-Provider ein.
- Du schützt alle Routen der Applikation mit einer einzigen Middleware-Datei.

9.1 Backend: User-Model & AuthController

Das `User` -Model speichert E-Mail-Adresse, Passwort-Hash und Anzeigename. Beachte: Das Feld heisst `PasswordHash` – niemals `Password` . Das Klartext-Passwort wird niemals in der Datenbank gespeichert.

Models/User.cs

```
public class User
{
    public int Id { get; set; }

    [Required, MaxLength(256)]
    public string Email { get; set; } = string.Empty;

    [Required]
    public string PasswordHash { get; set; } = string.Empty;

    [MaxLength(100)]
    public string DisplayName { get; set; } = string.Empty;
}
```

Der `AuthController` stellt zwei Endpunkte bereit:

Methode	Pfad	Funktion
POST	<code>/auth/register</code>	Neuen Benutzer anlegen (E-Mail + Passwort)
POST	<code>/auth/login</code>	Benutzer prüfen und Benutzerdaten zurückgeben

Kein JWT im Backend nötig!

Da wir NextAuth.js im Frontend verwenden, generiert das Backend *keine* JWT-Tokens. Der `/auth/login`-Endpunkt prüft nur Passwort und E-Mail und gibt bei Erfolg die Benutzerdaten zurück. NextAuth erzeugt daraus selbst einen signierten JWT für die Browser-Session. Das vereinfacht das Backend erheblich.

Clean Code B-06 — Culture-sicherer E-Mail-Vergleich

Ein häufiger Fehler: `.ToLower()` ist vom Betriebssystem-Locale abhängig. Auf einem Server mit türkischer Locale wird `"I".ToLower()` zu `"ı"` (Punkt-loses i) statt `"i"` — der E-Mail-Vergleich schlägt dann fehl (*Das türkische İ/i-Problem*).

✗ **Locale-abhängig** `u.Email.ToLower() == req.Email.ToLower()`

✓ **In LINQ / EF Core** `u.Email.ToUpperInvariant() == req.Email.ToUpperInvariant()`

✓ **In C#-Code** `u.Email.Equals(req.Email,
StringComparison.OrdinalIgnoreCase)`

Invariant bedeutet: locale-unabhängig, immer gleiche Gross-/Kleinschreibungsregel. In EF-Core-LINQ-Queries (die zu SQL übersetzt werden) funktioniert `Equals(...OrdinalIgnoreCase)` nicht direkt — deshalb `ToUpperInvariant()` verwenden.

9.2 Passwort-Hashing (ASP.NET Identity)

ASP.NET Core enthält bereits einen sicheren Passwort-Hasher als Teil des Identity-Frameworks – ohne zusätzliche NuGet-Pakete. Er verwendet **PBKDF2 mit HMAC-SHA256**, einem kryptographisch sicheren Algorithmus mit zufälligem Salt und Stretching.

Program.cs (Registrierung des Hashers)

```
// PasswordHasher für User registrieren  
builder.Services.AddScoped<IPasswordHasher<User>, PasswordHasher<User>>();
```

Controllers/AuthController.cs – Register-Endpunkt (Ausschnitt)

```
[HttpPost("register")]
public async Task<IActionResult> Register([FromBody] RegisterRequest req)
{
    // Prüfen ob E-Mail bereits existiert
    if (await _db.Users.AnyAsync(u => u.Email == req.Email))
        return Conflict(new { message = "E-Mail bereits registriert." });

    var user = new User {
        Email = req.Email,
        DisplayName = req.DisplayName ?? req.Email,
        PasswordHash = _hasher.HashPassword(null!, req.Password)
    };
    _db.Users.Add(user);
    await _db.SaveChangesAsync();

    // PasswordHash wird NIEMALS in der Antwort zurückgegeben!
    return Created($"/auth/{user.Id}",
        new { user.Id, user.Email, user.DisplayName });
}
```

Controllers/AuthController.cs – Login-Endpunkt (Ausschnitt)

```
[HttpPost("login")]
public async Task<IActionResult> Login([FromBody] LoginRequest req)
{
    var user = await _db.Users.FirstOrDefaultAsync(u => u.Email == req.Email);

    // TIMING-ATTACK-SCHUTZ: Passwort immer prüfen, auch wenn User nicht gefunden
    var hashZuPruefen = user?.PasswordHash ?? "dummy-hash";
    var result = _hasher.VerifyHashedPassword(null!, hashZuPruefen, req.Password);

    if (user == null || result == PasswordVerificationResult.Failed)
        return Unauthorized(new { message = "Ungültige Anmeldedaten." });

    return Ok(new { user.Id, user.Email, user.DisplayName });
}
```

Was ist ein Timing-Angriff?

Ein Angreifer kann messen, wie lange eine Anfrage dauert. Wenn `Unauthorized` bei einer unbekannten E-Mail *schneller* zurückkehrt als bei einer bekannten (weil das Passwort-Hashing übersprungen wird), kann der Angreifer daraus schliessen, ob eine E-Mail-Adresse im System existiert. Wir verhindern das, indem wir das Passwort *immer* hashen und vergleichen – auch wenn der User nicht gefunden wurde.

9.3 Frontend: NextAuth.js einrichten

NextAuth.js ist eine Open-Source-Bibliothek, die Authentication in Next.js standardisiert. Sie bietet fertige Provider (Google, GitHub, etc.) und ein flexibles Credentials-System. Wir installieren sie mit:

Terminal

```
cd frontend
npm install next-auth@^4.24.7
```

Die zentrale Konfiguration erfolgt in `lib/auth.ts`. Diese Datei exportiert `authOptions`, das von NextAuth verwendet wird:

lib/auth.ts (Struktur)

```
import type { NextAuthOptions } from 'next-auth';
import CredentialsProvider from 'next-auth/providers/credentials';
import GoogleProvider from 'next-auth/providers/google';

const INTERNAL_API = process.env.INTERNAL_API_URL ?? 'http://localhost:5000';

export const authOptions: NextAuthOptions = {
  providers: [
    CredentialsProvider({
      name: 'E-Mail & Passwort',
      credentials: { email: {...}, password: {...} },
      async authorize(credentials) {
        // Backend /auth/login aufrufen
        const response = await fetch(`${INTERNAL_API}/auth/login`, { ... });
        if (!response.ok) return null;
        const user = await response.json();
        return { id: String(user.id), email: user.email, name: user.displayName };
      },
    }),
    GoogleProvider({
      clientId: process.env.GOOGLE_CLIENT_ID ?? '',
      clientSecret: process.env.GOOGLE_CLIENT_SECRET ?? '',
    }),
  ],
  pages: { signIn: '/login' },
  session: { strategy: 'jwt', maxAge: 8 * 60 * 60 },
  secret: process.env.NEXTAUTH_SECRET,
};
```

Der NextAuth-Handler wird als Route Handler registriert. Alle Anfragen an `/api/auth/*` werden von NextAuth verarbeitet:

```
app/api/auth/[...nextauth]/route.ts
```

```
import NextAuth from 'next-auth';
import { authOptions } from '@lib/auth';

const handler = NextAuth(authOptions);
export { handler as GET, handler as POST };
```

Da `SessionProvider` ein Client-Hook verwendet, muss er in einer `'use client'`-Komponente liegen. Wir erstellen dafür einen dünnen `Providers`-Wrapper, den `layout.tsx` (Server Component) einbindet:

```
app/providers.tsx
```

```
'use client';
import { SessionProvider } from 'next-auth/react';

export default function Providers({ children }: { children: React.ReactNode }) {
  return <SessionProvider>{children}</SessionProvider>;
}
```

9.4 Google OAuth integrieren

Um Google als Login-Provider zu nutzen, muss in der [Google Cloud Console](#) ein OAuth-2.0-Client erstellt werden. Gehe dazu zu *APIs & Services* → *Anmeldedaten* → *OAuth 2.0-Client-IDs* und wähle «Webanwendung». Als *Authorized Redirect URI* muss eingetragen werden:

```
Google Cloud Console → Authorized redirect URIs
```

```
http://localhost:3000/api/auth/callback/google
```

Die generierten `GOOGLE_CLIENT_ID` und `GOOGLE_CLIENT_SECRET` werden in `.env.local` eingetragen:

```
.env.local
```

```
GOOGLE_CLIENT_ID=deine-client-id.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=dein-client-secret
```

Sicherheitshinweis: `.env.local` nie ins Git-Repository!

Die Datei `.env.local` enthält geheime Schlüssel und darf *niemals* in ein Git-Repository eingcheckedt werden. Stelle sicher, dass `.env.local` in der `.gitignore`-Datei eingetragen ist. Next.js macht das standardmässig automatisch.

9.5 Login-Seite & Route-Schutz

Die Login-Seite (`app/login/page.tsx`) zeigt ein Formular für E-Mail/Passwort und eine Schaltfläche für Google. Sie ruft `signIn()` von NextAuth auf. Der Parameter `redirect: false` verhindert, dass NextAuth automatisch weiterleitet – wir wollen den Fehlerfall selbst behandeln.

`app/login/page.tsx` (Ausschnitt)

```
const result = await signIn('credentials', {
  email,
  password: passwort,
  redirect: false,
  callbackUrl,
});

if (result?.error) {
  setFehler('Ungültige E-Mail-Adresse oder falsches Passwort.');
```

```
} else {
  router.push(callbackUrl);
}
```

Der Route-Schutz erfolgt über eine einzelne Middleware-Datei im Wurzelverzeichnis des Frontends. Die Middleware prüft bei jeder Anfrage, ob eine gültige NextAuth-Session vorhanden ist. Ohne Session wird der Benutzer zur Login-Seite umgeleitet.

`middleware.ts`

```
export { default } from 'next-auth/middleware';

export const config = {
  // Alle Routen schützen ausser /login und statische Assets
  matcher: ['/(?!(?!api/auth|_next/static|_next/image|favicon\\.ico|login).*)'],
};
```

Wie funktioniert NextAuth-Middleware?

NextAuth exportiert eine fertige `withAuth` -Middleware. Durch das Re-Exportieren als Default-Export (`export { default } from 'next-auth/middleware'`) wird sie für alle Routen aktiviert, die dem `matcher` -Muster entsprechen. Der `matcher` ist ein Regex-Muster: `(?!api/auth|_next/static|...)` ist eine *negative Lookahead-Gruppe* – sie schließt bestimmte Pfade vom Schutz aus.

Zusammenfassung Kapitel 9

- Passwörter werden mit PBKDF2/HMAC-SHA256 gehasht und niemals im Klartext gespeichert.
- Der `IPasswordHasher<T>` aus `Microsoft.AspNetCore.Identity` ist ohne zusätzliche NuGet-Pakete verfügbar.
- NextAuth.js vereinheitlicht verschiedene Login-Methoden (Credentials, Google, etc.) hinter einer einzigen API.
- Ein dünner `Providers` -Client-Wrapper ermöglicht den `SessionProvider` in einem Server-Component-Layout.
- Eine einzige `middleware.ts` schützt alle Routen mit einem Regex-Pattern.

KAPITEL 10

10. E-Mail-Benachrichtigung & Einstellungen

Lernziele dieses Kapitels

- Du implementierst einen SMTP-E-Mail-Dienst als Dependency-Injection-Service.
- Du verstehst das Interface-Muster in C# und warum es für Testbarkeit wichtig ist.
- Du speicherst App-Einstellungen als Datenbankeintrag und verwaltest sie über eine API.
- Du erweiterst den Adressen-Controller um eine E-Mail-Benachrichtigung beim Erstellen.

10.1 Backend: EmailService (SMTP)

Der E-Mail-Versand folgt einem Zwei-Schicht-Muster: Ein domänenspezifisches `INotificationService` -Interface kennt die Applikationsdomäne (z.B. «neue Adresse erfasst»), während das technische `IEmailService` -Interface die SMTP-Details kapselt. Dadurch bleibt der Controller schlank und die Notification-Logik ist wiederverwendbar (B-03).

Services/INotificationService.cs (B-03)

```
namespace AdressverwaltungApi.Services;

public interface INotificationService
{
    Task NotifyNewAdresseAsync(Adresse adresse);
}
```

Services/EmailNotificationService.cs

```
public class EmailNotificationService : INotificationService
{
    private readonly IEmailService _email;
    private readonly AdresseDbContext _db;

    public EmailNotificationService(IEmailService email, AdresseDbContext db)
    {
        _email = email;
        _db = db;
    }

    public async Task NotifyNewAdresseAsync(Adresse adresse)
    {
        var settings = await _db.Settings.FirstOrDefaultAsync();
        var notifEmail = settings?.NotificationEmail;
        if (string.IsNullOrEmpty(notifEmail)) return;

        await _email.SendAsync(
            notifEmail,
            "Neue Adresse erfasst",
            $"Neue Adresse: {adresse.Vorname} {adresse.Name}, " +
            $"{adresse.Strasse} {adresse.Strassennummer}, {adresse.Plz} {adresse.Ort}"
        );
    }
}
```

Das darunter liegende `IEmailService` ist der rein technische SMTP-Transport. Er liest Port und Absender-Adresse aus der Konfiguration — mit `int.TryParse` statt `int.Parse` (B-04) und dem einheitlichen Schlüssel `FromAddress` statt früher `SenderEmail` (B-05):

Services/IEmailService.cs

```
public interface IEmailService
{
    Task SendAsync(string to, string subject, string body);
}
```

Services/EmailService.cs (B-04, B-05)

```

public class EmailService : IEmailService
{
    private readonly IConfiguration _config;
    public EmailService(IConfiguration config) => _config = config;

    public async Task SendAsync(string to, string subject, string body)
    {
        var host      = _config["SmtpSettings:Host"]      ?? "localhost";
        var user      = _config["SmtpSettings:Username"] ?? "";
        var password  = _config["SmtpSettings:Password"] ?? "";

        // B-04: int.TryParse statt int.Parse (kein Absturz bei ungültigem Wert)
        if (!int.TryParse(_config["SmtpSettings:Port"], out var port))
            port = 587;

        // B-05: Einheitlicher Schlüssel "FromAddress" (früher "SenderEmail")
        var sender = _config["SmtpSettings:FromAddress"] ?? user;

        using var client = new SmtpClient(host, port)
        {
            Credentials = new NetworkCredential(user, password),
            EnableSsl    = true,
        };
        var message = new MailMessage(sender, to, subject, body);
        await client.SendMailAsync(message);
    }
}

```

int.Parse vs. int.TryParse (B-04)

`int.Parse` wirft eine `FormatException`, wenn der Wert in `appsettings.json` kein gültiger Integer ist — und stoppt damit den gesamten Application-Start. `int.TryParse` gibt `false` zurück und ermöglicht einen sicheren Fallback:

✗ **Absturz:** `var port = int.Parse(_config["SmtpSettings:Port"] ?? "587");`

✓ **Sicher:** `if (!int.TryParse(_config["SmtpSettings:Port"], out var port)) port = 587;`

Die SMTP-Einstellungen werden aus `appsettings.json` gelesen und können über Umgebungsvariablen überschrieben werden (wichtig für Docker). Der Schlüssel `FromAddress` muss in Code und `docker-compose.yml` identisch sein (B-05):

`appsettings.json` (Ausschnitt)

```
"SmtpSettings": {
  "Host": "smtp.example.com",
  "Port": 587,
  "Username": "user@example.com",
  "Password": "ERSETZE_MIT_ECHTEM_PASSWORT",
  "FromAddress": "noreply@example.com"
}
```

In `Program.cs` werden beide Services registriert:

Program.cs (Ausschnitt)

```
builder.Services.AddScoped<IEmailService, EmailService>();
builder.Services.AddScoped<INotificationService, EmailNotificationService>();
```

Im `AdressenController` wird nun der `INotificationService` injiziert — die SMTP-Details sind vollständig gekapselt. Der Controller hat keinerlei Kenntnis davon, wie die Benachrichtigung technisch funktioniert:

Controllers/AdressenController.cs – PostAdresse (Ausschnitt)

```
public async Task<IActionResult> PostAdresse([FromBody] Adresse adresse)
{
    _db.Adressen.Add(adresse);
    await _db.SaveChangesAsync();

    try { await _notificationService.NotifyNewAdresseAsync(adresse); }
    catch (Exception ex)
    {
        _logger.LogWarning("Benachrichtigung fehlgeschlagen: {Msg}", ex.Message);
    }

    return Created($"odata/Adressen({adresse.Id})", adresse);
}
```

Warum den E-Mail-Fehler nicht weiterwerfen?

Der E-Mail-Versand ist eine Nebenfunktion — die Adresse wurde bereits gespeichert. Würden wir den E-Mail-Fehler weiterwerfen, würde der Client fälschlicherweise einen 500-Fehler erhalten. Deshalb: Exception abfangen und nur protokollieren. Dieses Muster heisst *Fire-and-forget mit Logging*.

10.2 Backend: Settings-Entität & Controller

Applikationseinstellungen speichern wir als einzelne Zeile in einer eigenen Tabelle. Das ist einfacher als eine Key-Value-Tabelle und genügt für unsere Anforderungen.

Models/Settings.cs

```
public class Settings
{
    public int Id { get; set; }
    public string NotificationEmail { get; set; } = string.Empty;
}
```

Der `SettingsController` stellt zwei Endpunkte bereit: `GET /settings` liest die Einstellungen (oder erstellt einen Standardeintrag, falls keiner existiert), `PUT /settings` speichert die Änderungen.

Controllers/SettingsController.cs (GET, Ausschnitt)

```
[HttpGet]
public async Task<IActionResult> Get()
{
    var settings = await _db.Settings.FirstOrDefaultAsync();
    if (settings == null)
    {
        settings = new Settings();
        _db.Settings.Add(settings);
        await _db.SaveChangesAsync();
    }
    return Ok(new { notificationEmail = settings.NotificationEmail });
}
```

10.3 Frontend: Einstellungen-Seite

Die Einstellungen-Seite ist eine Client-Komponente, die beim ersten Rendern die aktuellen Einstellungen lädt und ein Formular zum Ändern der Benachrichtigungs-E-Mail anzeigt. Sie speichert die Änderungen per `PUT /settings` und zeigt eine Erfolgs- oder Fehlermeldung.

app/einstellungen/page.tsx (Ausschnitt)

```
// Einstellungen beim ersten Rendern laden
useEffect(() => {
  getSettings()
    .then((s) => setEmail(s.notificationEmail ?? ''))
    .catch(() => setFehler('Einstellungen konnten nicht geladen werden.'))
    .finally(() => setLaden(false));
}, []);

async function handleSubmit(e: React.FormEvent) {
  e.preventDefault();
  await speichereSettings({ notificationEmail: email.trim() });
  setErfolg(true);
}
```

Die Navigationsleiste (`NavBar`) verlinkt zur Einstellungen-Seite und zeigt den angemeldeten Benutzer sowie einen «Abmelden»-Button. Sie ist eine Client-Komponente, die `useSession()` von NextAuth verwendet.

Zusammenfassung Kapitel 10

- Services werden hinter Interfaces definiert, um Austauschbarkeit und Testbarkeit zu gewährleisten.
- SMTP-Einstellungen werden aus `appsettings.json` gelesen und können per Umgebungsvariable überschrieben werden.
- E-Mail-Fehler werden als Nebenfunktion behandelt: Logging statt Exception-Weiterwurf.
- Applikationseinstellungen als Datenbank-Entität sind einfach zu lesen, zu schreiben und zu versionieren.

KAPITEL 11

11. Sicherheitsüberlegungen

Lernziele dieses Kapitels

- Du kennst die häufigsten Sicherheitsprobleme bei Webanwendungen mit Authentifizierung.
- Du verstehst Timing-Angriffe und kannst sie mit konstantem Zeitaufwand abwehren.
- Du weisst, wie User-Enumeration verhindert wird.
- Du wendest Best Practices für Secrets und Umgebungsvariablen an.

11.1 Timing-Angriffe abwehren

Ein **Timing-Angriff** (engl. *timing attack*) ist eine Form des Seitenkanalangriffs, bei der ein Angreifer aus der Antwortzeit einer Anfrage Rückschlüsse auf interne Systemzustände zieht.

Beim Login: Wenn das System bei einer unbekannten E-Mail-Adresse sofort antwortet (ohne Passwort-Hashing), bei einer bekannten aber erst nach dem Hashing-Vorgang (~50 ms), kann der Angreifer durch Zeitmessungen herausfinden, ob eine E-Mail-Adresse im System existiert.

Lösung: Das Passwort-Hashing wird *immer* durchgeführt, auch wenn der Benutzer nicht existiert. Wir verwenden dazu einen Dummy-Hash:

AuthController.cs – Timing-Attack-Schutz

```
var user = await _db.Users.FirstOrDefaultAsync(u => u.Email == req.Email);

// IMMER hashen, auch wenn kein User gefunden wurde
var hashZuPruefen = user?.PasswordHash ?? "dummy-hash";
var result = _hasher.VerifyHashedPassword(null!, hashZuPruefen, req.Password);

// Nur wenn BEIDES stimmt (User existiert UND Passwort korrekt)
if (user == null || result == PasswordVerificationResult.Failed)
    return Unauthorized(...);
```

11.2 User-Enumeration verhindern

User-Enumeration bedeutet, dass ein Angreifer herausfinden kann, ob eine bestimmte E-Mail-Adresse in einem System registriert ist. Das geschieht häufig über unterschiedliche Fehlermeldungen: «*E-Mail nicht gefunden*» vs. «*Falsches Passwort*».

Lösung in unserem Login-Endpunkt: Wir geben in beiden Fällen die gleiche neutrale Fehlermeldung zurück:

AuthController.cs – neutrale Fehlermeldung

```
// Kein Unterschied zwischen "User nicht gefunden" und "falsches Passwort"
return Unauthorized(new { message = "Ungültige Anmeldedaten." });
```

Lösung im Register-Endpunkt: Hier ist es schwieriger – wenn wir sagen «*E-Mail bereits vorhanden*», ist das eine Enumeration. Für ein Tutorial ist das akzeptabel. In einer Produktions-App würde man stattdessen immer «*Falls die E-Mail noch nicht registriert ist, wurde ein Bestätigungslink gesendet*» zurückgeben – unabhängig davon, ob die E-Mail existiert.

11.3 Secrets & Umgebungsvariablen

Geheime Schlüssel dürfen niemals direkt im Quellcode stehen. Wir verwenden Umgebungsvariablen und die Datei `.env.local`:

Variable	Beschreibung	Beispielwert
<code>NEXTAUTH_SECRET</code>	JWT-Signierschlüssel für NextAuth	Ausgabe von <code>openssl rand -base64 32</code>
<code>NEXTAUTH_URL</code>	Öffentliche URL der App	<code>http://localhost:3000</code>
<code>GOOGLE_CLIENT_ID</code>	OAuth 2.0 Client-ID von Google	<code>xxx.apps.googleusercontent.com</code>
<code>GOOGLE_CLIENT_SECRET</code>	OAuth 2.0 Client-Secret von Google	Aus Google Cloud Console
<code>SmtpSettings__Password</code>	SMTP-Passwort (Docker env var)	App-Passwort des Mail-Anbieters

Ein sicherer `NEXTAUTH_SECRET` kann einfach per Terminal generiert werden:

Terminal

```
openssl rand -base64 32
```

Goldene Regeln für Secrets

- **Niemals** Secrets in Git-Repositories einchecken (auch nicht private!)
- **Niemals** Secrets in Kommentaren, Log-Ausgaben oder API-Antworten ausgeben
- Für jede Umgebung (Lokal, Staging, Produktion) **eigene** Secrets verwenden
- Secrets regelmässig **rotieren** (austauschen)
- `.env.local` und `appsettings.*.json` mit echten Werten gehören in `.gitignore`

Zusammenfassung Kapitel 11

- Timing-Angriffe werden durch konstanten Zeitaufwand (immer hashen) verhindert.
- Neutrale Fehlermeldungen verhindern, dass Angreifer existierende E-Mail-Adressen erkennen.

- Secrets werden ausschliesslich über Umgebungsvariablen und `.env.local` verwaltet – nie im Quellcode.
- Der `NEXTAUTH_SECRET` muss ein kryptographisch sicherer Zufallswert sein.

KAPITEL 12

12. Audit-Felder & Gültigkeitszeitraum

In produktiven Anwendungen muss nachvollziehbar sein, *wer* einen Datensatz *wann* erstellt oder geändert hat. Zusätzlich benötigen viele Geschäftsprozesse ein *Gültigkeitsdatum*: Eine Adresse, ein Preis oder eine Einstellung soll ab einem bestimmten Tag gelten und kann zu einem späteren Zeitpunkt ablaufen. In diesem Kapitel erweitern wir alle Entitäten um diese Felder und implementieren das automatische Befüllen im Backend.

12.1 Warum Audit-Trails?

Ein **Audit-Trail** (Revisionsprotokoll) hält fest:

- Wann wurde ein Datensatz **erstellt**?
- Von welchem Benutzer wurde er erstellt (`CreatedBy`)?
- Wann wurde er zuletzt **geändert**?
- Von welchem Benutzer wurde er geändert (`ChangedBy`)?

Audit-Trails sind in vielen Branchen gesetzlich vorgeschrieben (z.B. DSGVO, ISO 27001, Buchführungspflicht) und ermöglichen bei Fehlern oder Datenschutzvorfällen die genaue Rekonstruktion des Sachverhalts.

Der **Gültigkeitszeitraum** (`DateFrom` / `DateTo`) ermöglicht:

- **Zeitgerechtigkeit**: Zeige den Stand der Daten zu einem beliebigen Zeitpunkt in der Vergangenheit.
- **Planung**: Neue Einträge können schon heute erstellt werden, gelten aber erst ab einem künftigen Datum.
- **Sanftes Löschen**: Statt einen Datensatz zu löschen, wird `DateTo` auf das heutige Datum gesetzt – der Eintrag bleibt historisch erhalten.

Design-Prinzip: Read-Only im Frontend

Audit-Felder (`createDate` , `createdBy` , `changeDate` , `changedBy`) werden **ausschliesslich vom Backend** gesetzt. Das Frontend übergibt sie nie beim POST/PATCH. Sie werden nur zur Anzeige verwendet – etwa in einer Detailansicht oder einem Protokoll.

12.2 Backend: AuditableEntity Basisklasse

Anstatt die Audit-Felder in jeder Entität zu wiederholen, erstellen wir eine **abstrakte Basisklasse** `AuditableEntity`. Alle Entitäten erben von ihr. Das ist das *DRY-Prinzip* (Don't Repeat Yourself) in der Praxis.

Models/AuditableEntity.cs

```
namespace AdressverwaltungApi.Models;

/// <summary>
/// Abstrakte Basisklasse für alle Entitäten mit Audit-Feldern
/// und Gültigkeitszeitraum.
/// </summary>
public abstract class AuditableEntity
{
    // — Audit: Erstellung —————
    /// <summary>Zeitstempel der Erstellung (UTC), automatisch gesetzt.</summary>
    public DateTime CreateDate { get; set; }

    /// <summary>Benutzername, der den Datensatz erstellt hat.</summary>
    public string CreatedBy { get; set; } = string.Empty;

    // — Audit: Änderung —————
    /// <summary>Zeitstempel der letzten Änderung (UTC).</summary>
    public DateTime? ChangeDate { get; set; }

    /// <summary>Benutzername der letzten Änderung.</summary>
    public string? ChangedBy { get; set; }

    // — Gültigkeitszeitraum —————
    /// <summary>Gültig ab (Standard: heutiges Datum).</summary>
    public DateOnly DateFrom { get; set; }

    /// <summary>Gültig bis. null = unbegrenzt gültig.</summary>
    public DateOnly? DateTo { get; set; }
}
```

Wichtig: `DateOnly` ist ein C#-Typ (seit .NET 6), der nur das Datum ohne Uhrzeit speichert. Npgsql (der PostgreSQL-Treiber) mappt ihn automatisch auf den PostgreSQL-Typ `date`. Für Zeitstempel mit Uhrzeit verwenden wir `DateTime` (gemappt auf `timestamp without time zone`).

Alle Modelle erben nun von `AuditableEntity`:

Models/Adresse.cs (vereinfacht)

```
public class Adresse : AuditableEntity
{
    public int Id { get; set; }
    public string Vorname { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    // ... weitere Felder ...
}
```

Models/City.cs (vereinfacht)

```
public class City : AuditableEntity
{
    public int Id { get; set; }
    public string PostalCode { get; set; } = string.Empty;
    public string CityName { get; set; } = string.Empty;
}
```

Das gleiche Muster gilt für `User` und `Settings` .

12.3 DbContext: automatisches Befüllen der Audit-Felder

Die Audit-Felder sollen **automatisch** befüllt werden – ohne dass der aufrufende Code sich darum kümmern muss. Der richtige Ort dafür ist der `DbContext` : Wir überschreiben `SaveChangesAsync()` und setzen die Felder kurz vor dem Speichern.

IHttpContextAccessor – Wer ist eingeloggt?

Um den Benutzernamen zu ermitteln, benötigt der DbContext Zugriff auf den aktuellen HTTP-Request. ASP.NET Core bietet dafür das Interface `IHttpContextAccessor` . Wir injizieren es per Constructor-Injection:

Data/AdresseDbContext.cs (Konstruktor)

```

public class AdresseDbContext : DbContext
{
    private readonly IHttpContextAccessor? _httpContextAccessor;

    public AdresseDbContext(
        DbContextOptions<AdresseDbContext> options,
        IHttpContextAccessor? httpContextAccessor = null)
        : base(options)
    {
        _httpContextAccessor = httpContextAccessor;
    }
    // ...
}

```

Damit DI den `IHttpContextAccessor` bereitstellt, muss er in `Program.cs` registriert werden:

Program.cs (Auszug)

```

// IHttpContextAccessor für Audit-Felder registrieren
builder.Services.AddHttpContextAccessor();

```

SaveChangesAsync – die Kernlogik

Data/AdresseDbContext.cs (SaveChangesAsync)

```

public override Task<int> SaveChangesAsync(
    CancellationToken cancellationToken = default)
{
    var now = DateTime.UtcNow;

    // Benutzernamen ermitteln (Priorität: JWT-Claim > Header X-User > "system")
    var claimUser = _httpContextAccessor?.HttpContext?
        .User?.Identity?.Name;
    var headerUser = _httpContextAccessor?.HttpContext?
        .Request.Headers["X-User"].FirstOrDefault();
    var currentUser = !string.IsNullOrEmpty(claimUser) ? claimUser
        : !string.IsNullOrEmpty(headerUser) ? headerUser
        : "system";

    foreach (var entry in ChangeTracker.Entries<AuditableEntity>())
    {
        switch (entry.State)
        {
            case EntityState.Added:
                entry.Entity.CreateDate = now;
                entry.Entity.CreatedBy = currentUser;
                // DateFrom auf heute setzen, falls nicht explizit gesetzt
                if (entry.Entity.DateFrom == default)
                    entry.Entity.DateFrom = DateOnly.FromDateTime(now);
                break;

            case EntityState.Modified:
                entry.Entity.ChangeDate = now;
                entry.Entity.ChangedBy = currentUser;
                // CreateDate und CreatedBy niemals überschreiben!
                entry.Property(e => e.CreateDate).IsModified = false;
                entry.Property(e => e.CreatedBy).IsModified = false;
                break;
        }
    }

    return base.SaveChangesAsync(cancellationToken);
}

```

Reihenfolge der Benutzerermittlung

1. **JWT-Claim** (`User.Identity.Name`): wenn NextAuth ein Token mit gewältigem Benutzer übergibt
2. **HTTP-Header** `X-User` : kann manuell vom Frontend gesetzt werden
3. **Fallback** `"system"` : wenn kein Benutzerkontext verfügbar ist (z.B. während Migrationen oder Server-seitigen Tasks)

CreateDate niemals überschreiben

Im `EntityState.Modified` -Fall setzen wir `entry.Property(...).IsModified = false` für `CreateDate` und `CreatedBy`. Damit verhindert EF Core, dass diese Felder beim UPDATE überschrieben werden – auch wenn ein Client manipulierte Werte sendet.

Konfiguration im OnModelCreating

Damit EF Core die Audit-Felder korrekt abbildet, werden sie zentral im `OnModelCreating` über eine Schleife konfiguriert – einmal für alle Entitäten, die von `AuditableEntity` erben:

Data/AdresseDbContext.cs (OnModelCreating, Auszug)

```
foreach (var entityType in modelBuilder.Model.GetEntityTypes())
{
    if (typeof(AuditableEntity).IsAssignableFrom(entityType.ClrType))
    {
        modelBuilder.Entity(entityType.ClrType, b =>
        {
            b.Property<DateTime>("CreateDate")
                .IsRequired()
                .HasColumnType("timestamp without time zone");
            b.Property<string>("CreatedBy")
                .IsRequired().HasMaxLength(100);
            b.Property<DateTime?>("ChangeDate")
                .HasColumnType("timestamp without time zone");
            b.Property<string?>("ChangedBy")
                .HasMaxLength(100);
            b.Property<DateOnly>("DateFrom")
                .IsRequired().HasColumnType("date");
            b.Property<DateOnly?>("DateTo")
                .HasColumnType("date");
        });
    }
}
```

12.4 Migration: neue Spalten hinzufügen

Da wir eine bestehende Datenbank erweitern (und nicht neu erstellen), erstellen wir eine neue inkrementelle Migration. Alle sechs Spalten werden für jede Tabelle hinzugefügt.

Migrations/20240101000002_AddAuditFields.cs (Auszug)

```

private static readonly string[] Tables =
    ["Adressen", "Cities", "Users", "Settings"];

protected override void Up(MigrationBuilder migrationBuilder)
{
    foreach (var table in Tables)
    {
        migrationBuilder.AddColumn<DateTime>(
            name: "CreateDate", table: table,
            type: "timestamp without time zone", nullable: false,
            defaultValueSql: "NOW() AT TIME ZONE 'UTC'");

        migrationBuilder.AddColumn<string>(
            name: "CreatedBy", table: table,
            type: "character varying(100)", maxLength: 100,
            nullable: false, defaultValue: "migration");

        migrationBuilder.AddColumn<DateTime>(
            name: "ChangeDate", table: table,
            type: "timestamp without time zone", nullable: true);

        migrationBuilder.AddColumn<string>(
            name: "ChangedBy", table: table,
            type: "character varying(100)", nullable: true);

        migrationBuilder.AddColumn<DateOnly>(
            name: "DateFrom", table: table,
            type: "date", nullable: false,
            defaultValue: new DateOnly(2024, 1, 1));

        migrationBuilder.AddColumn<DateOnly>(
            name: "DateTo", table: table,
            type: "date", nullable: true);
    }
}

```

Standardwerte für bestehende Zeilen

Bestehende Datenbankzeilen erhalten beim Ausführen der Migration automatisch Standardwerte:

- `CreateDate` → aktueller UTC-Zeitstempel (`NOW() AT TIME ZONE 'UTC'`)
- `CreatedBy` → `"migration"` (Platzhalter, da kein Benutzer bekannt)
- `DateFrom` → `2024-01-01`
- `ChangeDate` , `ChangedBy` , `DateTo` → `NULL`

Da in `Program.cs` `db.Database.Migrate()` beim Start ausgeführt wird, wird die Migration automatisch beim nächsten Start der Applikation (z.B. via Docker Compose) auf die Datenbank angewendet.

Datenbankstruktur nach der Migration

Spalte	Typ (PostgreSQL)	Nullable	Beschreibung
CreateDate	timestamp without time zone	NOT NULL	Zeitstempel der Erstellung (UTC)
CreatedBy	character varying(100)	NOT NULL	Erstellt von (Benutzername)
ChangeDate	timestamp without time zone	NULL	Zeitstempel der letzten Änderung (UTC)
ChangedBy	character varying(100)	NULL	Zuletzt geändert von
DateFrom	date	NOT NULL	Gültig ab (Standard: heute)
DateTo	date	NULL	Gültig bis (null = unbegrenzt)

12.5 Frontend: TypeScript-Typen anpassen

Im Frontend spiegeln wir die neuen Felder in den TypeScript-Typen. Wir folgen dem gleichen DRY-Prinzip wie im Backend: Die Audit-Felder werden in einem gemeinsamen Interface `AuditFields` definiert und von allen Entitäts-Interfaces geerbt.

types/auditable.ts

```
/**
 * Audit-Felder - geerbt von allen Backend-Entitäten.
 * Werden ausschliesslich vom Backend gesetzt (read-only im Frontend).
 * ISO 8601 Strings (wie von OData / JSON geliefert).
 */
export interface AuditFields {
  createDate: string; // "2024-06-01T12:30:00Z"
  createdBy: string;
  changeDate: string | null;
  changedBy: string | null;
  dateFrom: string; // "2024-06-01"
  dateTo: string | null;
}
```

types/adresse.ts

```
import { AuditFields } from './auditable';

export interface Adresse extends AuditFields {
  id: number;
  vorname: string;
  name: string;
  strasse: string;
  strassennummer: string;
  plz: string;
  ort: string;
}

// Beim Erstellen: Audit-Felder und id weglassen
// dateFrom/dateTo sind optional (Backend setzt Standardwerte)
export type AdresseCreate = Omit<Adresse,
  'id' | 'createDate' | 'createdBy' | 'changeDate' | 'changedBy'
> & {
  dateFrom?: string;
  dateTo?: string | null;
};

// Beim Aktualisieren: alle Felder optional
export type AdresseUpdate = Partial<Omit<Adresse,
  'id' | 'createDate' | 'createdBy' | 'changeDate' | 'changedBy'
>>;
```

Das gleiche Muster gilt für `types/city.ts`: `City` erbt von `AuditFields`, und `CityCreate` lässt die Audit-Felder weg.

OData und camelCase

OData-Antworten verwenden durch `EnableLowerCamelCase()` in `Program.cs` automatisch camelCase. Daher heisst `CreateDate` in der JSON-Antwort `createDate` – genau wie im TypeScript-Interface definiert.

Audit-Felder im Frontend anzeigen (Beispiel)

Audit-Felder können einfach in einer Detailansicht angezeigt werden. Da `createDate` ein ISO-8601-String ist, wandeln wir ihn mit dem JavaScript-Konstruktor `new Date()` um:

Beispiel: Audit-Info in einer React-Komponente

```
function AuditInfo({ entity }: { entity: AuditFields }) {
  const format = (iso: string | null) =>
    iso ? new Date(iso).toLocaleString('de-CH') : '-';

  return (
    <div className="audit-info">
      <p>Erstellt am: {format(entity.createDate)} von {entity.createdBy}</p>
      {entity.changeDate && (
        <p>Geändert am: {format(entity.changeDate)} von {entity.changedBy}</p>
      )}
      <p>Gültig von: {entity.dateFrom}
        {entity.dateTo ? ` bis ${entity.dateTo}` : ' (unbegrenzt)'}
      </p>
    </div>
  );
}
```

Zusammenfassung Kapitel 12

- Alle Entitäten erben von der abstrakten Klasse `AuditableEntity` – DRY-Prinzip.
- `CreateDate` / `CreatedBy` werden automatisch beim INSERT gesetzt.
- `ChangeDate` / `ChangedBy` werden automatisch bei jedem UPDATE gesetzt.
- Das Überschreiben von `CreateDate` beim UPDATE wird mit `IsModified = false` verhindert.
- Der Benutzername wird über `IHttpContextAccessor` ermittelt (JWT-Claim > Header > "system").
- `DateFrom` / `DateTo` ermöglichen zeitliche Gültigkeit und sanftes Löschen.
- Die neue Migration befüllt bestehende Zeilen mit sinnvollen Standardwerten.
- Im Frontend werden die Audit-Felder über ein gemeinsames Interface `AuditFields` getypt.

KAPITEL 13

13. Häufige Fehler & Debugging

Lernziele dieses Kapitels

- Du erkennst, warum ein Seed-Benutzer wegen NOT-NULL-Constraints scheitern kann.
- Du verstehst, was den `OAuthSignin`-Fehler in NextAuth auslöst und wie du ihn behebst.
- Du kannst den Google-Login-Button gezielt ein- und ausblenden.
- Du weisst, wann du das Docker-Volume neu erstellen und den Browser-Cache löschen musst.

Beim Erweitern einer laufenden Applikation tauchen immer wieder dieselben Problemmuster auf. Dieses Kapitel dokumentiert die wichtigsten Stolpersteine und zeigt, wie du sie systematisch erkennst und behebst.

13.1 Seed-Benutzer schlägt fehl (NOT NULL)

Das Problem

Nachdem wir in Kapitel 12 die `AuditableEntity`-Basisklasse einführten, erbt das `User`-Model — wie alle anderen Entitäten — von ihr. Das bedeutet: die Felder `CreateDate`, `CreatedBy` und `DateFrom` sind NOT NULL. Wenn der Seed-Code in `Program.cs` einen Benutzer anlegt und dabei `db.SaveChanges()` aufruft, schreibt EF Core die Felder *nicht* automatisch — der INSERT scheitert mit einem PostgreSQL-Fehler wie:

Fehler (PostgreSQL-Log)

```
ERROR: null value in column "CreatedBy" of relation "Users" violates not-null constraint
```

Der Benutzer wird *nicht* in der Datenbank angelegt. Alle nachfolgenden Login-Versuche scheitern mit HTTP 401, weil der Account schlicht nicht existiert.

Die Ursache

Im `AdresseDbContext` hatten wir nur `SaveChangesAsync()` überschrieben, um `ApplyAuditFields()` aufzurufen. Den synchronen `SaveChanges()` hatten wir vergessen. EF Core ruft je nach Context *entweder* die synchrone *oder* die asynchrone Variante auf — den Seed in `Program.cs` ruft die **synchrone** Variante auf.

Die Lösung

Beide Methoden müssen überschrieben werden. Wir extrahieren zusätzlich eine private Hilfsmethode `ApplyAuditFields()`, damit der Code nur einmal steht (DRY-Prinzip):

Data/AdresseDbContext.cs – beide SaveChanges-Overrides

```

// Extrahierte Hilfsmethode - wird von beiden Overrides aufgerufen
private void ApplyAuditFields()
{
    var user = _httpContextAccessor?.HttpContext?.User?.Identity?.Name
               ?? _httpContextAccessor?.HttpContext?.Request.Headers["X-
User"].FirstOrDefault()
               ?? "system";

    var now = DateTime.UtcNow;

    foreach (var entry in ChangeTracker.Entries<AuditableEntity>())
    {
        if (entry.State == EntityState.Added)
        {
            entry.Entity.CreateDate = now;
            entry.Entity.CreatedBy = user;
            entry.Entity.DateFrom = DateOnly.FromDateTime(now);
        }
        if (entry.State == EntityState.Modified)
        {
            entry.Entity.ChangeDate = now;
            entry.Entity.ChangedBy = user;
            // CreateDate / CreatedBy nie überschreiben
            entry.Property(e => e.CreateDate).IsModified = false;
            entry.Property(e => e.CreatedBy).IsModified = false;
        }
    }
}

// Synchroner Override - wird z.B. beim Seed in Program.cs aufgerufen
public override int SaveChanges()
{
    ApplyAuditFields();
    return base.SaveChanges();
}

// Asynchroner Override - wird von API-Controllern aufgerufen
public override Task<int> SaveChangesAsync(CancellationToken ct = default)
{
    ApplyAuditFields();
    return base.SaveChangesAsync(ct);
}

```

Wichtig: Beide Varianten überschreiben

EF Core stellt `SaveChanges()` und `SaveChangesAsync()` als separate virtuelle Methoden bereit. Wer nur eine davon überschreibt, riskiert, dass in bestimmten Codepfaden die Audit-Logik stillschweigend übersprungen wird. **Immer beide überschreiben!**

13.2 NextAuth: OAuthSignin-Fehler & 401

Symptom

Die Login-Seite zeigt nach dem Klick auf «Anmelden» keine Fehlermeldung, leitet aber auf `/login?error=OAuthSignin` um. Im Browser-Netzwerk-Tab sieht man einen `401 Unauthorized`-Response vom NextAuth-Endpunkt.

Ursache 1: Leere Google-Credentials

NextAuth registriert den `GoogleProvider` auch dann, wenn `GOOGLE_CLIENT_ID` und `GOOGLE_CLIENT_SECRET` leere Strings sind. Beim ersten Seitenaufruf versucht NextAuth, den CSRF-State für *alle* registrierten Provider zu initialisieren. Ein Provider mit leeren Credentials schlägt dabei fehl und korrumpiert den CSRF-Cookie — was anschliessend sogar den Credentials-Login blockiert.

Lösung: GoogleProvider nur in die Provider-Liste aufnehmen, wenn echte Credentials konfiguriert sind:

lib/auth.ts

```
const googleClientId    = process.env.GOOGLE_CLIENT_ID    ?? '';
const googleClientSecret = process.env.GOOGLE_CLIENT_SECRET ?? '';
const googleEnabled      = googleClientId.length > 0 && googleClientSecret.length > 0;

export const authOptions: NextAuthOptions = {
  providers: [
    CredentialsProvider({ /* ... */ }),

    // Google nur aktivieren, wenn echte Credentials gesetzt sind
    ...(googleEnabled
      ? [GoogleProvider({ clientId: googleClientId, clientSecret: googleClientSecret })]
      : []),
  ],
  // ...
};
```

Ursache 2: Falscher Feldname in der Backend-Antwort

Der Credentials-Provider in `auth.ts` ruft nach erfolgreichem Login das `authorize()`-Callback auf. Dieses liest die JSON-Antwort des Backends und gibt ein User-Objekt zurück. Wenn das Backend den Benutzernamen als `name` liefert, aber der Code versucht `user.displayName` zu lesen, ist der `name`-Wert im Token `undefined` — NextAuth verweigert daraufhin die Session.

lib/auth.ts - authorize()-Callback (korrekt)

```

async authorize(credentials) {
  // ...
  const user = await response.json() as {
    id: string;
    name: string; // <-- Backend liefert "name", NICHT "displayName"
    email: string;
  };

  return {
    id: String(user.id),
    email: user.email,
    name: user.name, // <-- korrekt: "name" lesen
  };
}

```

Tipp: Backend-Antwort im Netzwerk-Tab prüfen

Öffne die Browser-Entwicklertools (F12) → Netzwerk-Tab, filtere nach `/auth/login` und schau dir die JSON-Antwort genau an. Vergleiche die Feldnamen mit dem, was `authorize()` liest. Schreibfehler wie `displayName` statt `name` sind ein klassischer Bug.

13.3 Google-Button bedingt anzeigen

Wenn Google OAuth nicht konfiguriert ist, sollte der Google-Button auf der Login-Seite gar nicht angezeigt werden. Da Umgebungsvariablen wie `GOOGLE_CLIENT_ID` serverseitig sind (kein `NEXT_PUBLIC_`-Prefix), kann eine `'use client'`-Komponente sie *nicht* lesen.

Die Lösung: Die Login-Seite wird in zwei Dateien aufgeteilt.

Datei	Typ	Aufgabe
<code>app/login/page.tsx</code>	Server Component	Liest <code>GOOGLE_CLIENT_ID</code> , übergibt <code>googleEnabled</code> als Prop
<code>app/login/LoginForm.tsx</code>	Client Component (<code>'use client'</code>)	Formular-Logik, empfängt <code>googleEnabled: boolean</code>

```
app/login/page.tsx (Server Component)
```

```
// Kein 'use client' → darf process.env lesen
import { Suspense } from 'react';
import { LoginForm } from './LoginForm';

export default function LoginPage() {
  const googleEnabled = !!(
    process.env.GOOGLE_CLIENT_ID?.trim() &&
    process.env.GOOGLE_CLIENT_SECRET?.trim()
  );

  return (
    <main className="min-h-screen flex items-center justify-center bg-gray-100">
      <Suspense fallback={<div>Wird geladen...</div>}>
        <LoginForm googleEnabled={googleEnabled} />
      </Suspense>
    </main>
  );
}
```

app/login/LoginForm.tsx (Client Component, Ausschnitt)

```

'use client';
import { useState } from 'react';
import { signIn } from 'next-auth/react';
import { useRouter, useSearchParams } from 'next/navigation';

interface LoginFormProps {
  googleEnabled: boolean;
}

export function LoginForm({ googleEnabled }: LoginFormProps) {
  // ... Formularzustand ...

  return (
    <div className="bg-white rounded-2xl shadow-lg w-full max-w-md p-8">
      {/* Credentials-Formular */}
      <form onSubmit={handleCredentials}>{/* ... */}</form>

      {/* Google-Abschnitt nur einblenden wenn OAuth konfiguriert */}
      {googleEnabled && (
        <>
          <div className="flex items-center gap-3 my-5">
            <div className="flex-1 border-t border-gray-200" />
            <span className="text-xs text-gray-400">oder</span>
            <div className="flex-1 border-t border-gray-200" />
          </div>
          <button onClick={handleGoogle}>Mit Google anmelden</button>
        </>
      )}
    </div>
  );
}

```

Warum Suspense?

`useSearchParams()` ist ein Client-Hook, der beim serverseitigen Rendering (SSR) einen Bail-out auslöst. Next.js 14 verlangt deshalb, dass jede Komponente, die `useSearchParams()` nutzt, in eine `<Suspense>`-Boundary eingebettet ist. Ohne diese Boundary schmeißt der Build einen Fehler: *«Missing Suspense boundary with useSearchParams»*.

13.4 Docker: Volume & Browser-Cache

Wann muss das Docker-Volume gelöscht werden?

Ein Docker-Volume enthält die persistenten Daten der PostgreSQL-Datenbank. Wenn EF Core Migrations ändern (neue Tabellen, geänderte Spalten) oder der Seed-Code geändert wurde, muss das Volume gelöscht werden, damit die Änderungen bei nächsten Start tatsächlich angewendet werden.

Terminal (im Projekt-Verzeichnis)

```
# Alle Container stoppen UND Volumes löschen (Daten werden gelöscht!)
docker compose down -v

# Container neu bauen und starten (Migrations + Seed werden neu ausgeführt)
docker compose up --build
```

Achtung: Alle Daten werden gelöscht

Das Flag `-v` löscht alle in `docker-compose.yml` definierten Volumes. Alle Testdaten und manuell angelegten Einträge gehen verloren. In einer Produktionsumgebung darf dieser Befehl nur mit ausdrücklicher Genehmigung ausgeführt werden.

Wann muss der Browser-Cache gelöscht werden?

NextAuth speichert zwei Cookies im Browser:

- `next-auth.csrf-token` – CSRF-Schutztoken (beim ersten Seitenaufruf erstellt)
- `next-auth.callback-url` – letzte Weiterleitungs-URL nach dem Login

Nach einem Docker-Rebuild mit `docker compose down -v` kann der alte CSRF-Cookie im Browser nicht mehr gegen das neu gestartete NextAuth validiert werden. Das führt zu einem `403 Forbidden` oder einem sofortigen Redirect auf `/login?error=...`.

Cookies im Browser löschen (Chrome / Edge)

Entwicklertools öffnen (F12) → *Application* → *Storage* → *Cookies* → `http://localhost:3000` auswählen → alle `next-auth.*`-Einträge löschen → Seite neu laden.

Alternativ: Im Adressfeld `localhost:3000` eingeben → links vom Schloss-Icon auf das Symbol klicken → *Cookies und Website-Daten löschen*.

Checkliste: Login funktioniert nicht?

Gehe diese Punkte der Reihe nach durch:

1. Backend läuft? → `curl http://localhost:5000/health` oder Swagger-UI öffnen.
2. Seed erfolgreich? → Log von `docker compose up` nach «Admin-Benutzer angelegt» durchsuchen.
3. Richtige E-Mail und Passwort? → Standard: `admin@example.com` / `Test-1234!`
4. NEXTAUTH_SECRET gesetzt? → `docker-compose.yml` prüfen (kein Platzhalter-Wert!)

5. Browser-Cookies gelöscht? → siehe oben
6. Docker-Volume neu erstellt? → `docker compose down -v && docker compose up --build`
7. Netzwerk-Tab im Browser: Welcher HTTP-Status? Was gibt `/api/auth/session` zurück?

Zusammenfassung Kapitel 13

- Beide `SaveChanges()` -Overrides müssen `ApplyAuditFields()` aufrufen — nur einen zu überschreiben reicht nicht.
- Den `GoogleProvider` nur in die NextAuth-Provider-Liste aufnehmen, wenn echte Credentials gesetzt sind — leere Credentials korrumpieren den CSRF-State.
- Den Feldnamen der Backend-Antwort (`name`) immer mit dem Code (`user.name`) abgleichen.
- Die Login-Seite in Server Component (`page.tsx`) und Client Component (`LoginForm.tsx`) aufteilen, um `process.env` serverseitig zu lesen.
- Nach einem `docker compose down -v` immer auch die Browser-Cookies für `localhost:3000` löschen.

KAPITEL 14

14. Clean Code & Refactoring

Lernziele dieses Kapitels

- Du kennst die Kernprinzipien von Clean Code (SRP, DRY, Fehlerbehandlung, Benennung).
- Du kannst alle 14 Befunde der Clean-Code-Analyse benennen und erklären.
- Du führst einen strukturierten Refactoring-Workflow durch: Analyse → Befunde → Massnahmen → Testen.

14.1 Was ist Clean Code?

Clean Code ist ein Begriff, der durch Robert C. Martins gleichnamiges Buch (2008) geprägt wurde. Das Ziel: Code soll nicht nur funktionieren, sondern auch *lesbar*, *wartbar* und *erweiterbar* sein. Denn Code wird weitaus öfter gelesen als geschrieben — gute Benennung, klare Struktur und konsistente Fehlerbehandlung zählen sich direkt in der Teamarbeit aus.

Die vier wichtigsten Prinzipien

Prinzip	Bedeutung	Beispiel aus unserem Projekt
SRP	Single Responsibility — jede Klasse hat genau eine Aufgabe	Seed-Code in <code>DbSeeder.cs</code> statt in <code>Program.cs</code>
DRY	Don't Repeat Yourself — Logik nur einmal definieren	Generischer <code>ODataCrudController<T></code> statt Duplikaten
Fehlerbehandlung	Robuste, vorhersehbare Fehlerantworten	<code>int.TryParse</code> statt <code>int.Parse</code>
Benennung	Namen beschreiben <i>Was</i> und <i>Warum</i>	<code>sucheLaeuft</code> statt <code>suchelaeuft</code>

Ein praktisches Kriterium: Code ist «clean», wenn ein Entwickler, der ihn zum ersten Mal liest, versteht was er tut — ohne Kommentare zu brauchen. Kommentare erklären das *Warum*, nicht das *Was*.

14.2 Übersicht: 14 Befunde

Die Clean-Code-Analyse unserer Adressverwaltungs-App ergab 14 Befunde — 8 im Backend (C#) und 6 im Frontend (TypeScript/React). Alle wurden in Version 2.0 behoben:

#	Priorität	Kategorie	Problem (vorher)	Datei	Massnahme (nachher)
B-01	Hoch	SRP	Seed-Code in <code>Program.cs</code>	<code>Program.cs</code>	→ <code>DbSeeder.cs</code>
B-02	Hoch	SRP	DTOs im Controller-File	<code>AuthController.cs</code>	→ <code>Dtos/</code> -Ordner
B-03	Mittel	SRP	E-Mail-Logik im Controller	<code>AdressenController.cs</code>	→ <code>INotificationService</code>
B-04	Hoch	Fehlerbehandlung	<code>int.Parse</code> ohne Schutz	<code>EmailService.cs</code>	→ <code>int.TryParse</code>
B-05	Hoch	Konfiguration	Inkonsistenter Schlüsselname	<code>EmailService.cs</code>	<code>SenderEmail</code> → <code>FromAddress</code>

B-06	Mittel	Fehlerbehandlung	<code>.ToLower()</code> nicht culture-sicher	<code>AuthController.cs</code>	→ <code>OrdinalIgnoreCase</code>
B-07	Mittel	Validierung	Fehlende DataAnnotations am Model	<code>Models/Adresse.cs</code>	<code>[Required]</code> , <code>[MaxLength]</code>
B-08	Mittel	DRY	CRUD-Duplikation in Controllern	beide Controller	→ generische Basisklasse
F-01	Mittel	DRY	Doppelte Audit-Normalisierung	<code>lib/api.ts</code>	→ <code>normalizeAuditFields</code>
F-02	Mittel	UI-Konsistenz	Controlled/Uncontrolled gemischt	<code>AdresseForm.tsx</code>	Alle Controlled mit <code>useState</code>
F-03	Hoch	Fehlerbehandlung	<code>confirm()</code> / <code>alert()</code>	<code>app/page.tsx</code>	→ <code>ConfirmDialog</code> - Komponente
F-04	Niedrig	TypeScript	Fehlende Session-Augmentation	<code>lib/auth.ts</code>	→ <code>types/next-auth.d.ts</code>
F-05	Niedrig	Benennung	<code>suchelaeuft</code> (Tippfehler)	<code>AdresseForm.tsx</code>	→ <code>sucheLaeuft</code>
F-06	Niedrig	Benennung	<code>ladeDaten</code> ohne <code>useCallback</code>	<code>app/page.tsx</code>	→ <code>useCallback(..., [])</code>

Wie viele Befunde sind normal?

14 Befunde in einer Lernprojekt-Codebasis sind vollkommen normal und sogar erwartet. Der Zweck der Analyse ist nicht Kritik, sondern Lernen: Jeder Befund ist eine Lerngelegenheit. In professionellen Code-Reviews lernt man, Befunde nach Priorität einzuordnen und systematisch abzuarbeiten.

14.3 Refactoring-Workflow

Ein strukturierter Refactoring-Workflow stellt sicher, dass Verbesserungen keine neuen Fehler einführen. Vier Phasen:

1. **Analyse:** Code lesen und Befunde dokumentieren (Datei, Zeile, Kategorie, Priorität). Noch nicht ändern.
2. **Priorisieren:** **Hoch** zuerst, dann **Mittel**, dann **Niedrig**. SRP-Verletzungen vor Style-Korrekturen.

3. **Massnahme:** Eine Änderung auf einmal. Datei erstellen/ändern, compilieren (`dotnet build`), dann testen.
4. **Validieren:** Nach jeder Änderung: Läuft die App noch? Bestehende Endpunkte antworten korrekt?

Refactoring-Schritte für das Backend (Beispiel)

```
# 1. Neue Datei anlegen (DbSeeder.cs)
# 2. Seed-Logik aus Program.cs verschieben
# 3. In Program.cs: DbSeeder.Seed(...) aufrufen, alten Code entfernen
# 4. Compilieren und testen

cd adressverwaltung/backend/AdressverwaltungApi
dotnet build          # Keine Compiler-Fehler?
dotnet run            # App startet?
# Swagger UI: http://localhost:5000/swagger
```

Niemals «big bang» refactoren

Ein häufiger Fehler: Alle Befunde auf einmal in einem grossen Commit beheben. Wenn danach etwas nicht mehr funktioniert, ist die Fehlersuche extrem schwierig. Besser: ein Befund pro Commit, mit einer kurzen Commit-Message wie `refactor: B-01 Seed-Logik in DbSeeder extrahiert`.

Zusammenfassung Kapitel 14

- Clean Code bedeutet: lesbar, wartbar, erweiterbar — nicht nur funktional.
- SRP, DRY, Fehlerbehandlung und saubere Benennung sind die vier Kernprinzipien.
- Die Adressverwaltungs-App hatte 14 Befunde (8 Backend + 6 Frontend) — alle wurden in Version 2.0 behoben.
- Refactoring immer schrittweise: ein Befund, ein Commit, dann testen.
- `int.TryParse` statt `int.Parse`, `OrdinalIgnoreCase` statt `.ToLower()` — kleine Änderungen, grosse Wirkung.